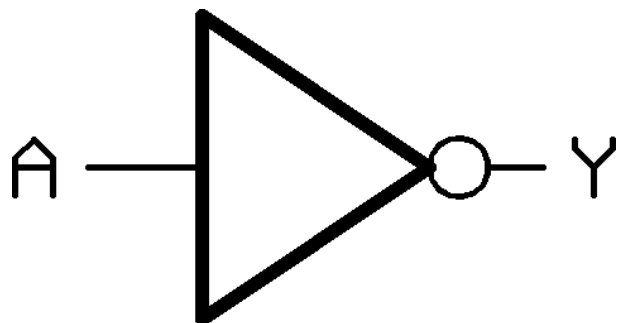
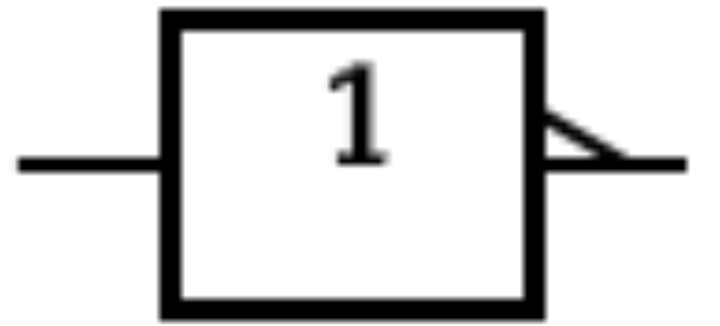


Portas Lógicas Básicas

Circuitos (Digitais) Combinacionais

Porta NOT (Inversora)

- Símbolo ANSI/IEEE: 

- Símbolo IEC: 

- Tabela Verdade:

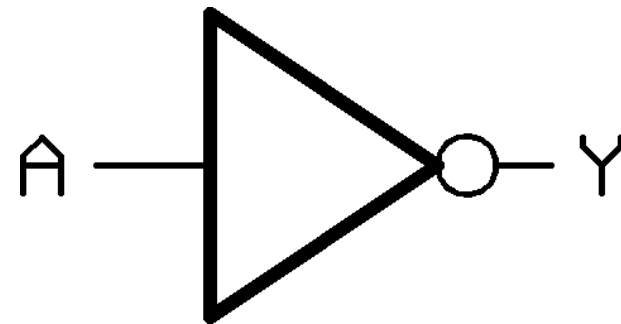
Entrada A	Saída Y
0	1
1	0

- Equação (booleana):

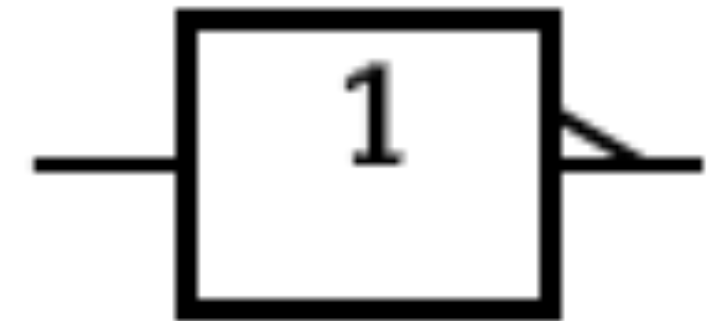
$$Y = \bar{A}$$

Porta NOT (Inversora)

- Símbolo ANSI/IEEE:



- Símbolo IEC:



Símbolos de **formato distintivo** (*distinctive-shape*)

IEEE Std 91

ANSI Y32.14

IEEE/AIEE 91-1962

Início: 1953 (diretriz militar MIL-STD_806)

Última atualização: **1984** (Reafirmado em **1994** no IEEE 91)

(Não é um padrão internacional, mas tolerado)

Símbolos de **formato retangular** (*rectangular-shape*):

ANSI/IEEE Std 91-1984

(IEEE Standard Graphics Symbols for Logic Functions, em conjunto com o American National Standards Institute)

Padrão internacional IEC 60617-12

(International Electrotechnical Commission)

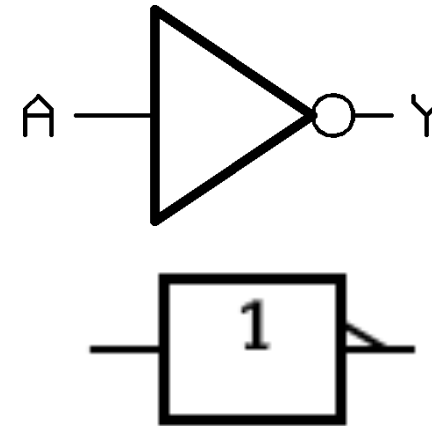
Início: 1984 (Como norma conjunta ANSI/IEEE Std 91-1984)

Última atualização: **1991** (Adendo IEEE 91a) / **1997** (IEC 60617-12)

Status Atual: Ambos os padrões estão maduros e não sofrem alterações profundas desde a década de 1990, pois a indústria migrou o design de circuitos para **linguagens de descrição de hardware (como VHDL e Verilog)**, tornando os diagramas de portas lógicas puramente educacionais ou restritos a blocos funcionais de alto nível.

Linguagens VHDL e Verilog...

VHDL: Mercado de FPGAs, mundo acadêmico, Setor militar, aeroespacial, Telecom, infra. Industrial



Maior adoção ASICs, Processadores

Verilog: Usa o operador de negação bit a bit ~, ou primitiva not:

```
-- Importa a biblioteca padrão para tipos de dados lógicos
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Define as entradas e saídas do componente
entity porta_not is
    Port (
        entrada : in  std_logic;
        saída   : out std_logic
    );
end porta_not;

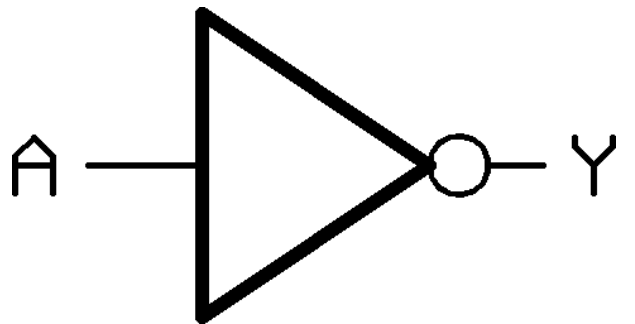
-- Define o comportamento interno do componente
architecture Comportamental of porta_not is
begin
    -- Atribuição concorrente (o sinal da saída é atualizado
    se a entrada mudar)
        saída <= not entrada;
    end Comportamental;
```

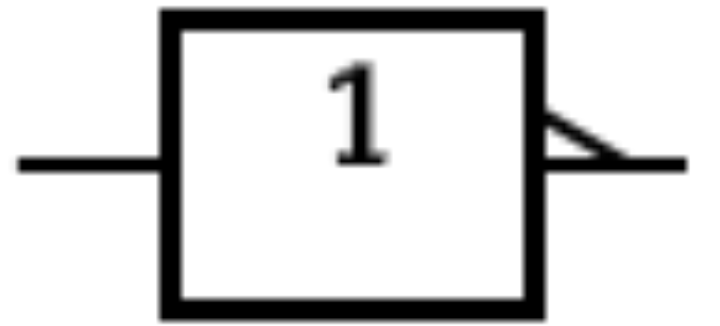
```
module porta_not (
    input  entrada,
    output saída
);
    // Atribuição contínua usando o operador bitwise NOT (~)
    assign saída = ~entrada;
endmodule
```

```
module porta_not_estrutural (
    input  entrada,
    output saída
);
    // Primitiva nativa: nome_da_porta (saída, entrada);
    not instancia1 (saída, entrada);
endmodule
```

SystemVerilog: versão expandida do Verilog (designs de CIs, ASICs — indústria).

Porta NOT (Inversora)

- Símbolo ANSI/IEEE: 

- Símbolo IEC: 

- Tabela Verdade:

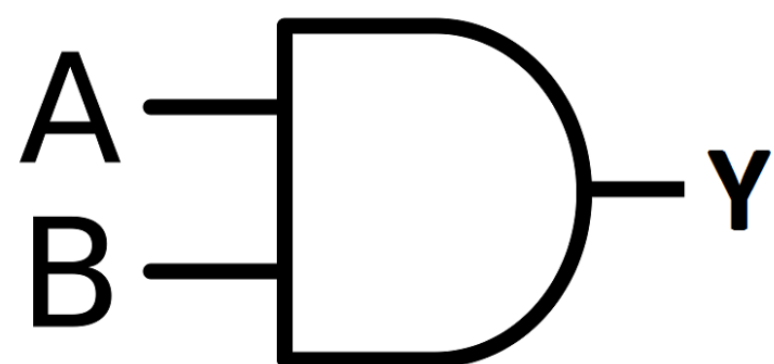
Entrada A	Saída Y
0	1
1	0

- Equação (booleana):

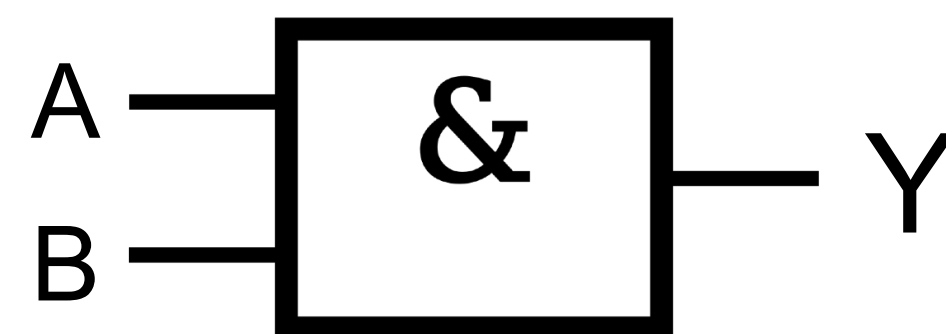
$$Y = \bar{A}$$

Porta AND

- Símbolo ANSI/IEEE:



- Símbolo IEC:



- Tabela verdade

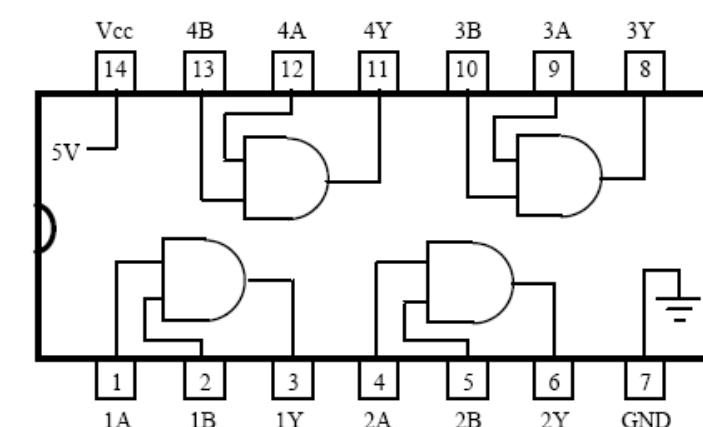
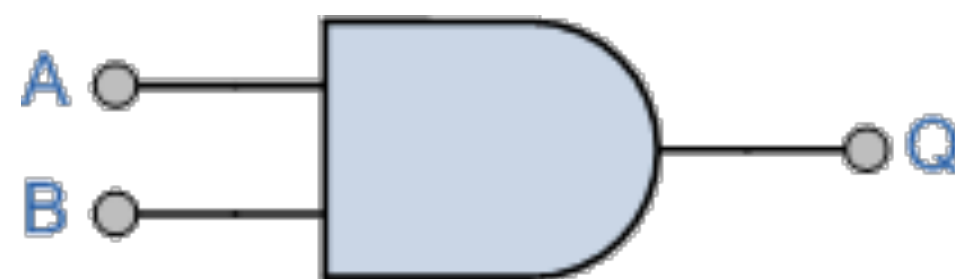
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Note: $x \cdot 0 = 0$
 $x \cdot 1 = x$
 $1 \cdot 1 = 1$

- Equação:

$$Y = A \cdot B$$

- Note:

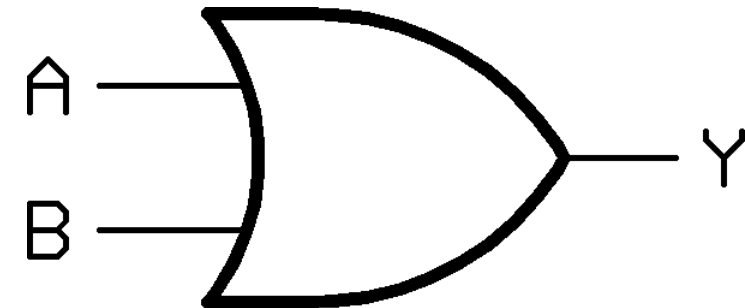


74LS08

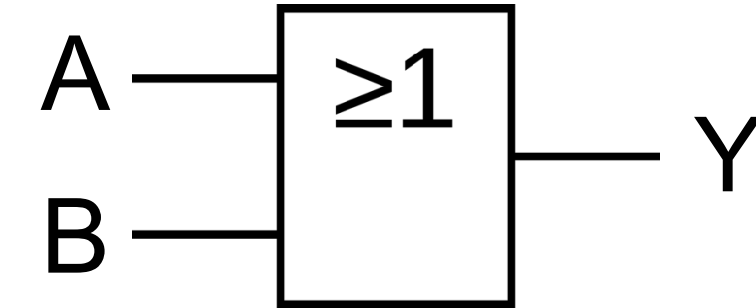
Obs.: entradas > 1 / saídas = 1.

Porta OR

- Símbolo ANSI/IEEE:



- Símbolo IEC:



- Tabela verdade

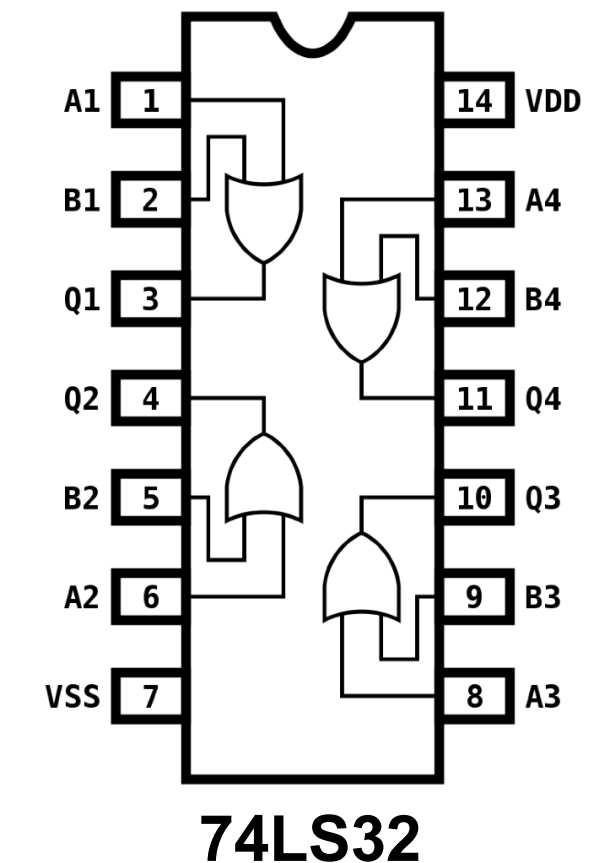
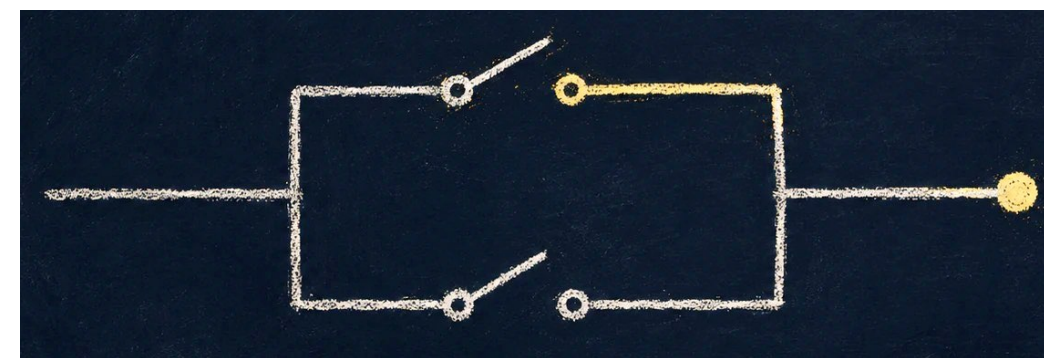
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Note: $x + 1 = 1$
 $x + 0 = x$
 $0 + 0 = 0$

- Equação:

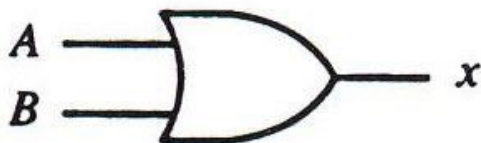
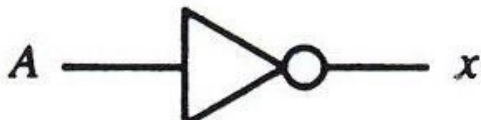
$$Y = A + B$$

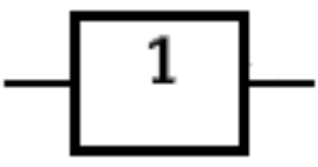
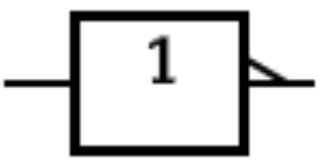
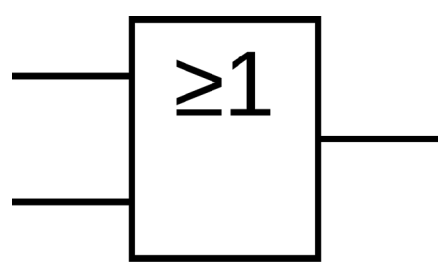
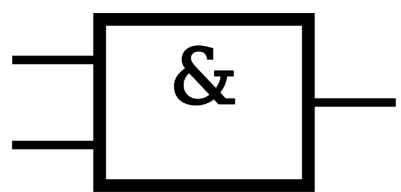
- Note:



Obs.: entradas > 1 / saídas = 1.

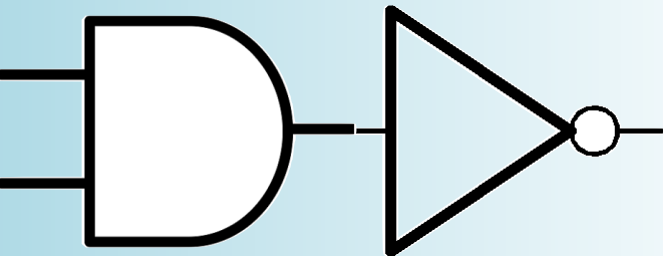
Primeiro Resumo

Name	Graphic symbol	Algebraic function	Truth table															
AND		$x = A \cdot B$ or $x = AB$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	0	0	1	0	1	0	0	1	1	1
A	B	x																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$x = A + B$	<table><tr><th>A</th><th>B</th><th>x</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	x	0	0	0	0	1	1	1	0	1	1	1	1
A	B	x																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$x = A'$	<table><tr><th>A</th><th>x</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	x	0	1	1	0									
A	x																	
0	1																	
1	0																	
Buffer		$x = A$	<table><tr><th>A</th><th>x</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	x	0	0	1	1									
A	x																	
0	0																	
1	1																	

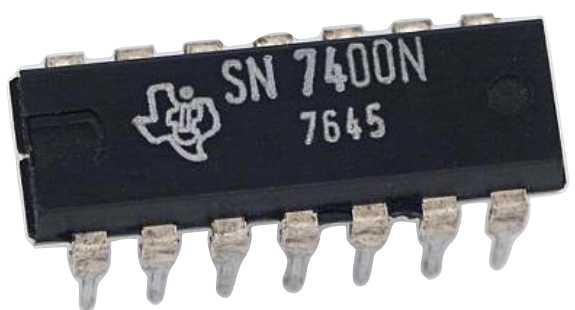
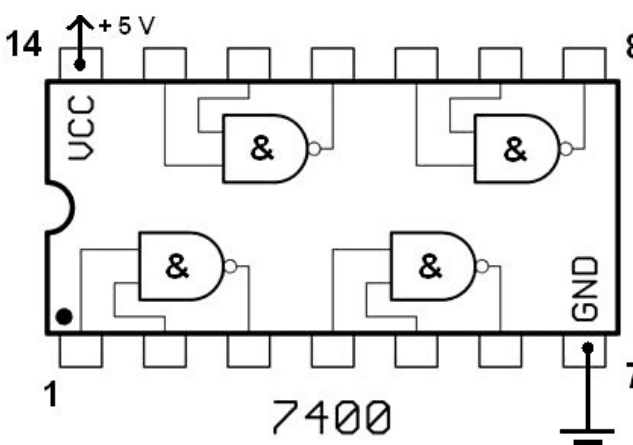
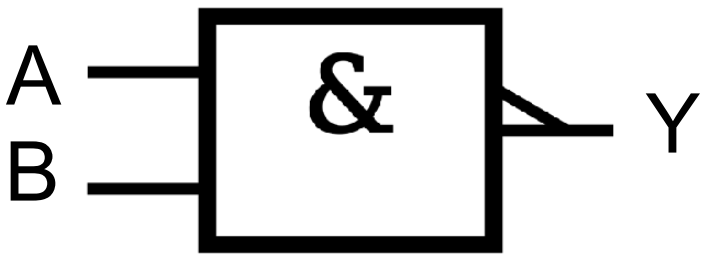
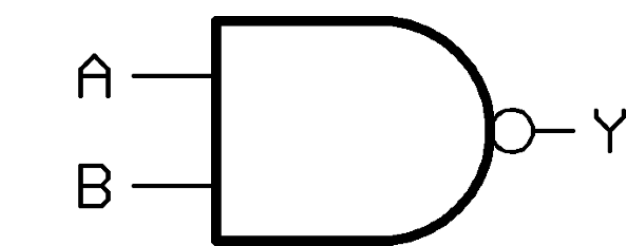


Variações

AND + NOT



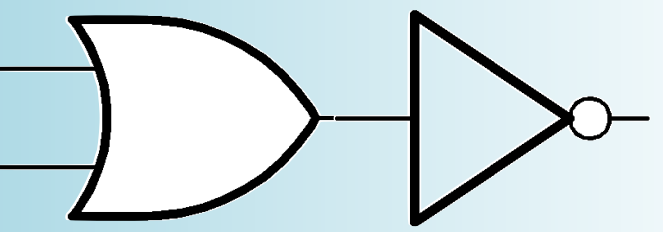
NAND



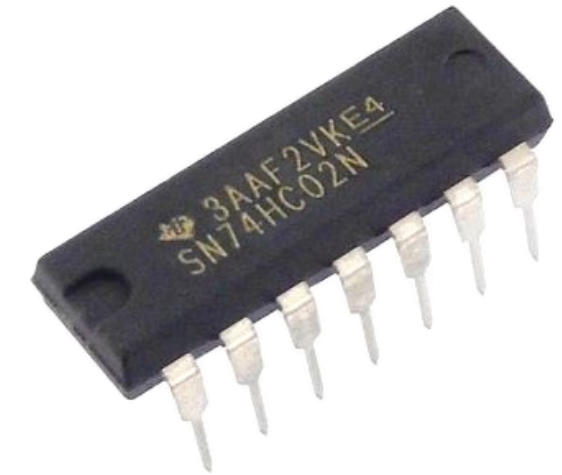
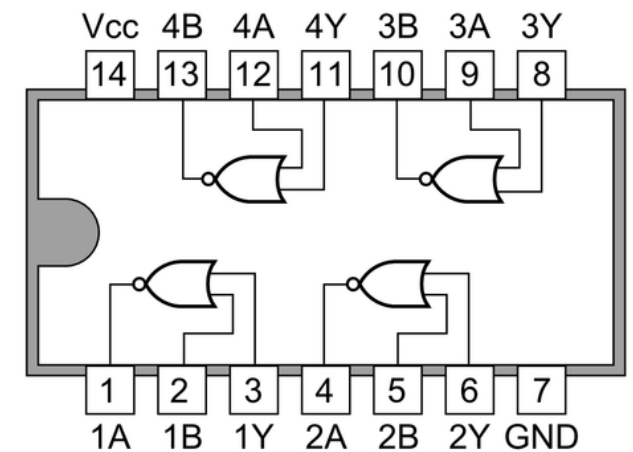
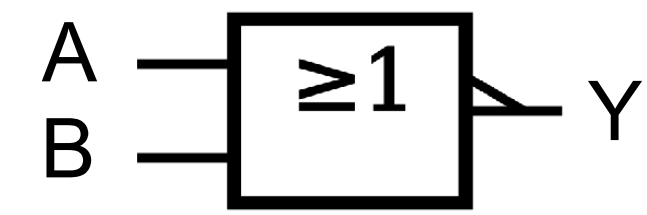
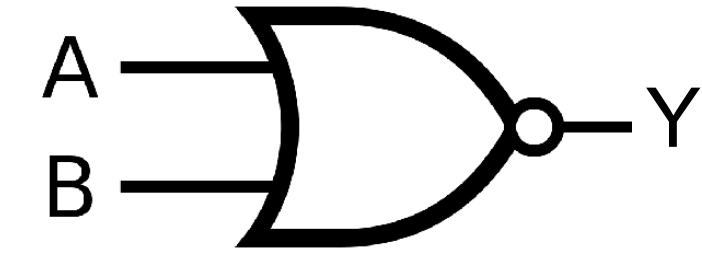
NAND

A B	AND $= A \cdot B$	NAND $Y = \overline{A \cdot B}$	Y
0 0	0	1	$Y = \overline{0 \cdot 0} = \overline{0} = 1$
0 1	0	1	$Y = \overline{0 \cdot 1} = \overline{0} = 1$
1 0	0	1	$Y = \overline{1 \cdot 0} = \overline{0} = 1$
1 1	1	0	$Y = \overline{1 \cdot 1} = \overline{1} = 0$

OR + NOT



NOR



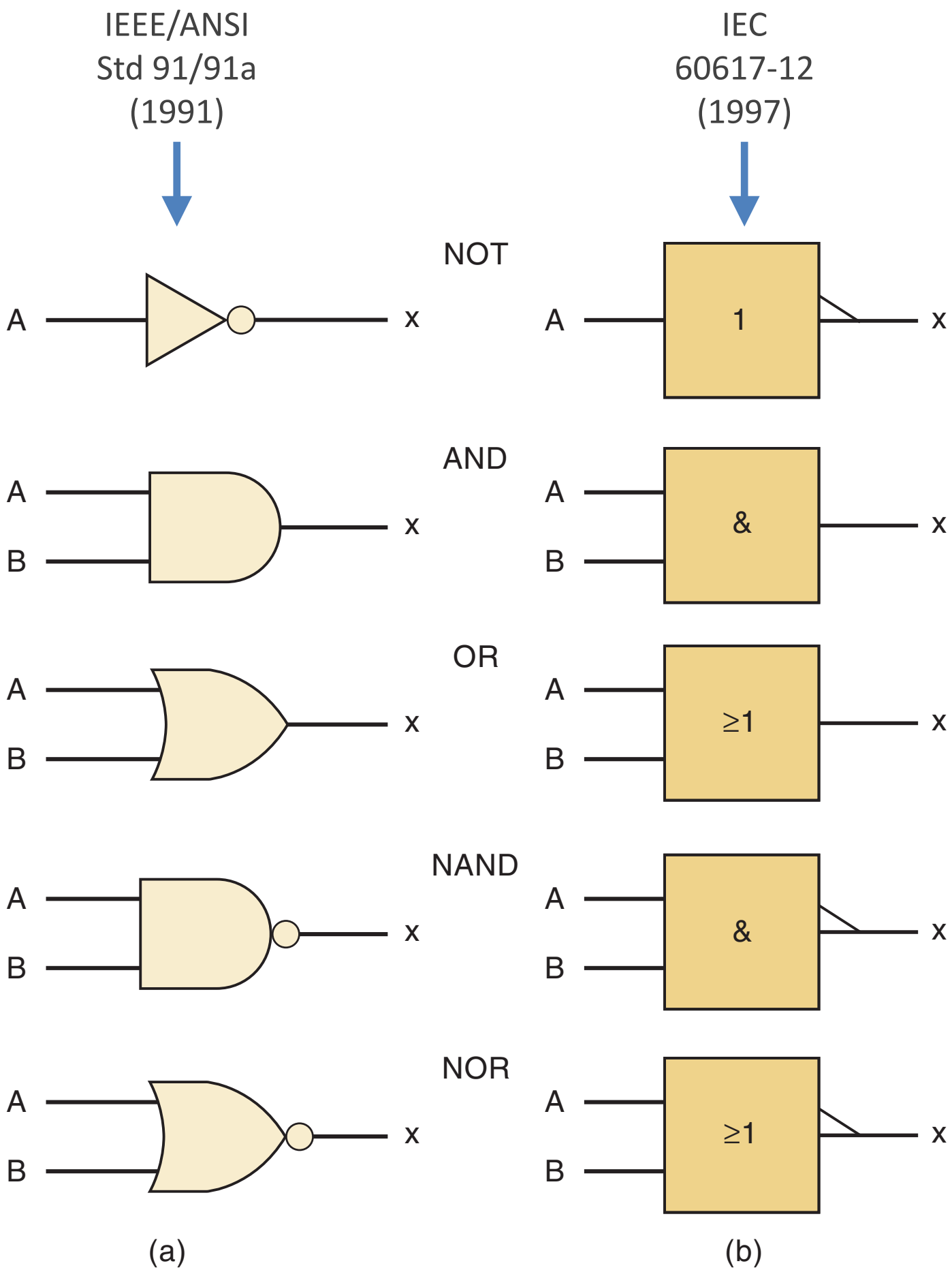
NOR

A B	OR $= A + B$	NOR $Y = \overline{A + B}$	Y
0 0	0	1	$Y = \overline{0 + 0} = \overline{0} = 1$
0 1	1	0	$Y = \overline{0 + 1} = \overline{1} = 0$
1 0	1	0	$Y = \overline{1 + 0} = \overline{1} = 0$
1 1	1	0	$Y = \overline{1 + 1} = \overline{1} = 0$

Resumo

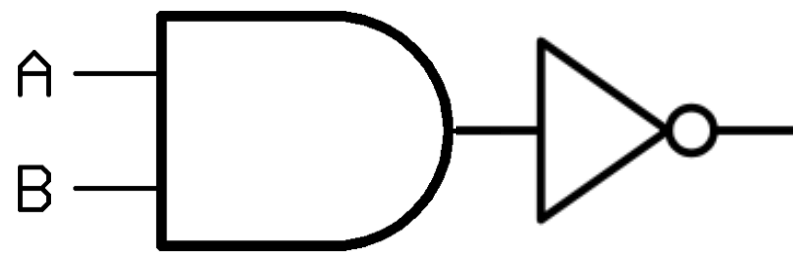
A B	AND	OR	NAND	NOR
0 0	0	0	1	1
0 1	0	1	1	0
1 0	0	1	1	0
1 1	1	1	0	0
	$x = A \cdot B$	$x = A + B$	$x = \overline{A \cdot B}$	$x = \overline{A + B}$

FIGURE 3-41 Standard logic symbols: (a) traditional; (b) IEEE/ANSI.

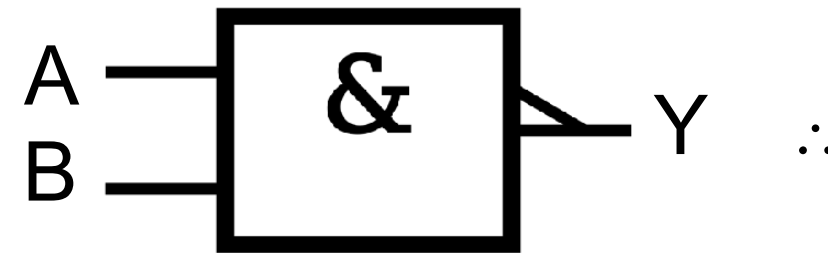
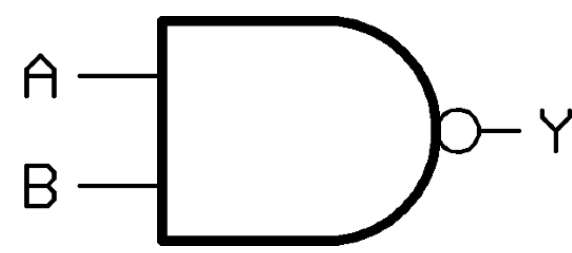


Atenção

AND + NOT



NAND



Equação NAND:

$$Y = \overline{A \cdot B}$$

NAND:

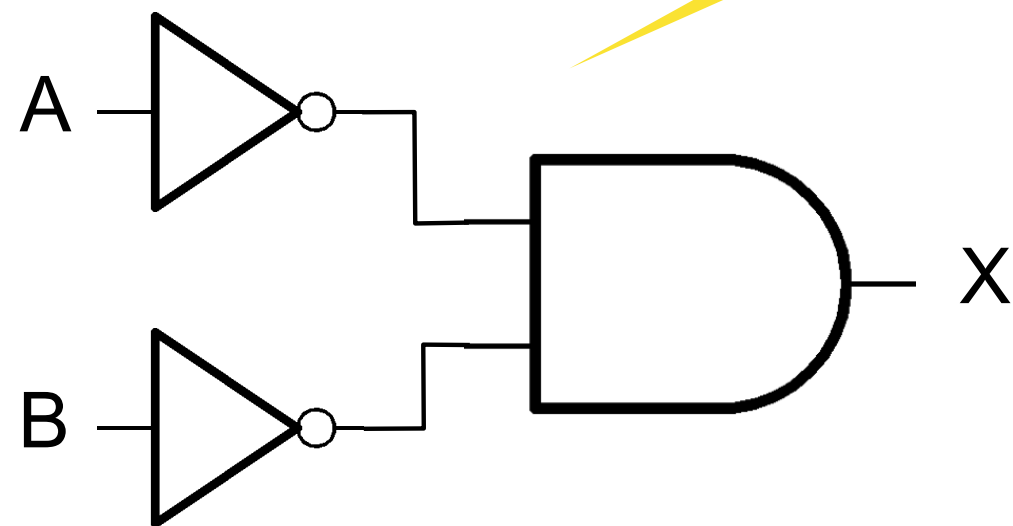
A	B	Y	AND
0	0	$Y = \overline{0 \cdot 0} = \overline{0} = 1$	0
0	1	$Y = \overline{0 \cdot 1} = \overline{0} = 1$	0
1	0	$Y = \overline{1 \cdot 0} = \overline{0} = 1$	0
1	1	$Y = \overline{1 \cdot 1} = \overline{1} = 0$	1

Note porém, que:

$$X = \overline{A} \cdot \overline{B}$$

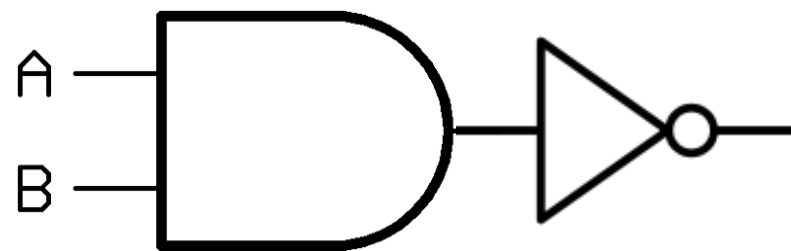
$\neq$

$$Y = \overline{A \cdot B}$$

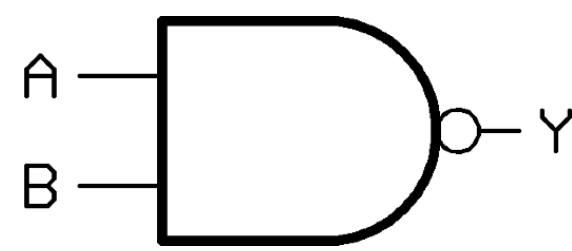


Atenção

AND + NOT



NAND



Equação NAND:

$$Y = \overline{A \cdot B}$$

NAND:

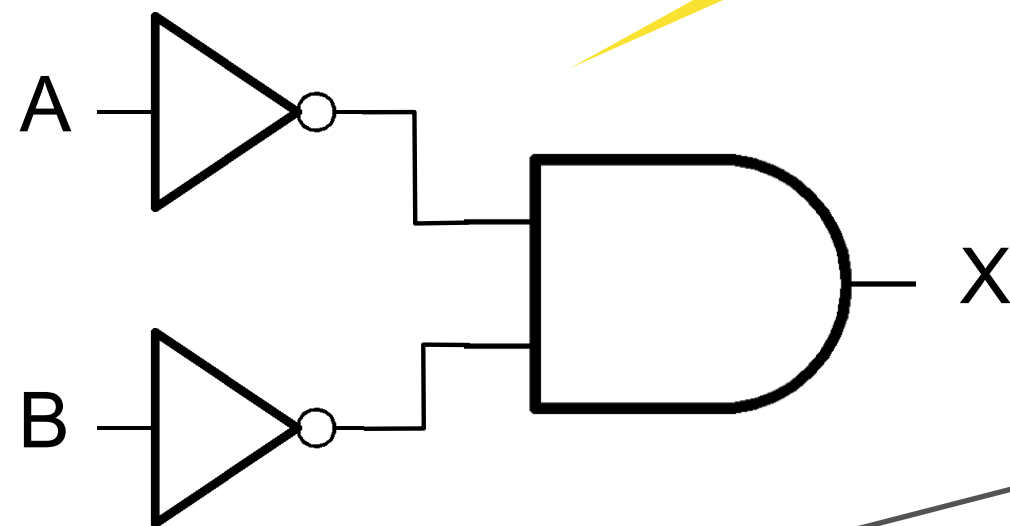
A	B	Y	AND
0	0	$Y = \overline{0 \cdot 0} = \overline{0} = 1$	0
0	1	$Y = \overline{0 \cdot 1} = \overline{0} = 1$	0
1	0	$Y = \overline{1 \cdot 0} = \overline{0} = 1$	0
1	1	$Y = \overline{1 \cdot 1} = \overline{1} = 0$	1

Note porém, que:

$$X = \overline{A} \cdot \overline{B}$$

$\neq$

$$Y = \overline{A \cdot B}$$

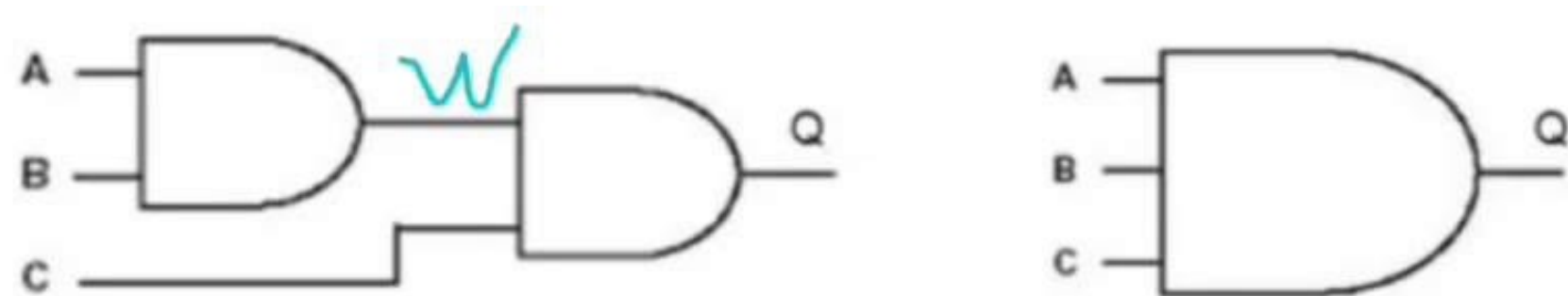


NOR !

A	B	X
0	0	$X = \overline{A} \cdot \overline{B} = \overline{0} \cdot \overline{0} = 1 \cdot 1 = 1$
0	1	$X = \overline{A} \cdot \overline{B} = \overline{0} \cdot \overline{1} = 1 \cdot 0 = 0$
1	0	$X = \overline{A} \cdot \overline{B} = \overline{1} \cdot \overline{0} = 0 \cdot 1 = 0$
1	1	$X = \overline{A} \cdot \overline{B} = \overline{1} \cdot \overline{1} = 0 \cdot 0 = 0$

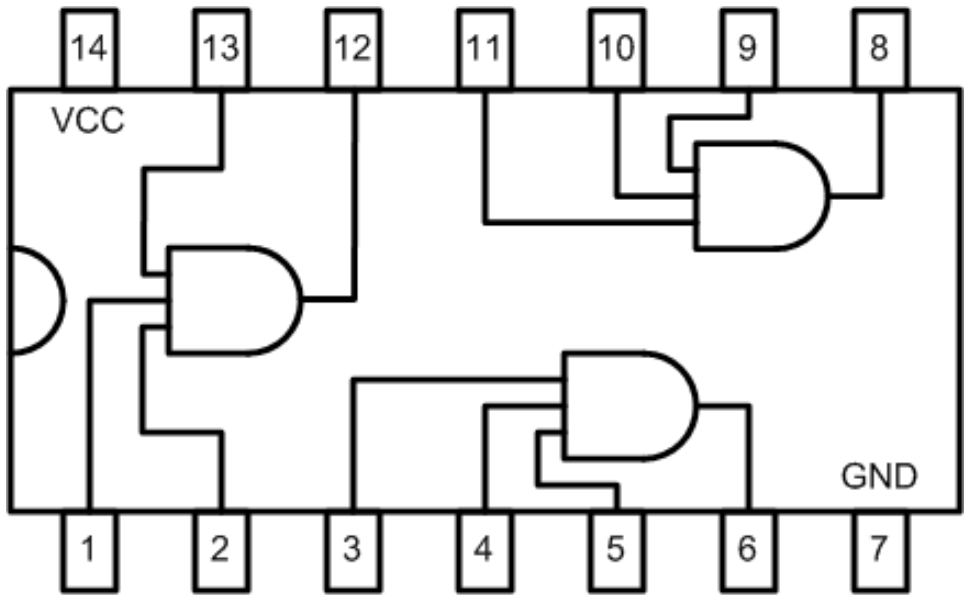
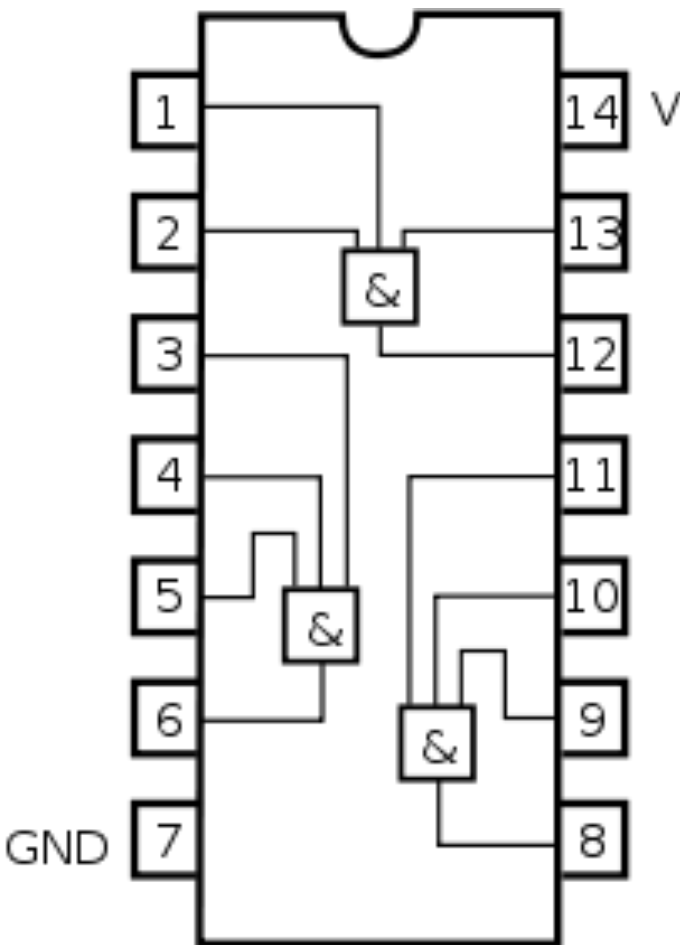
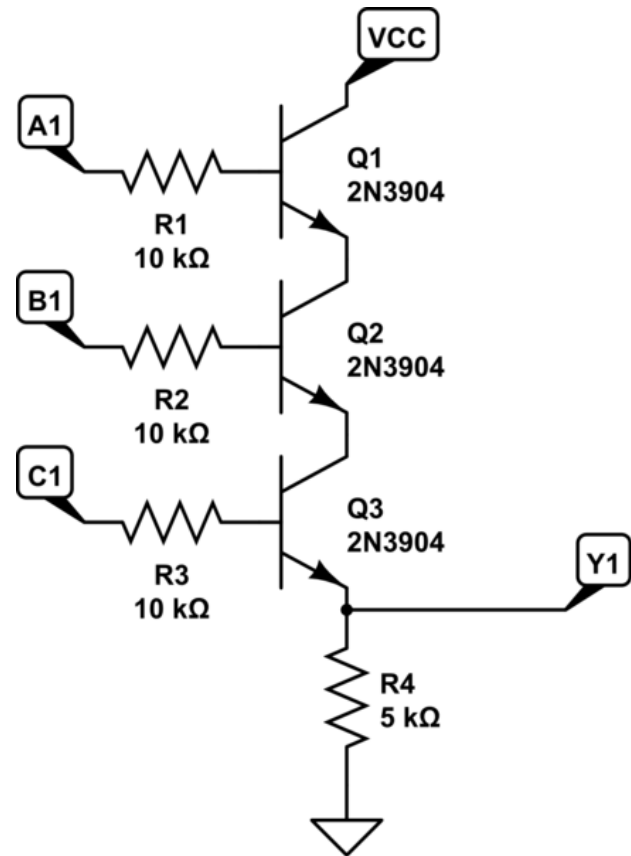
$\neq$

Análise Circuitos: Exemplos:



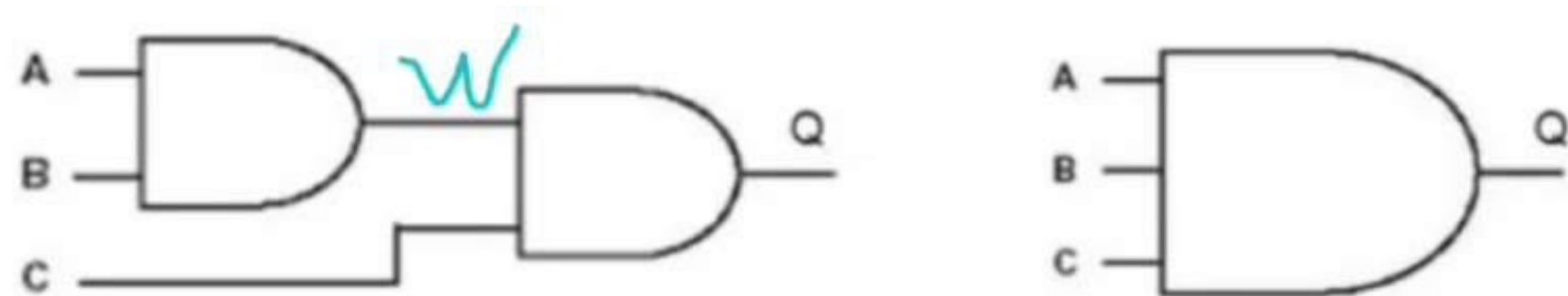
Equação ?

3 input AND				
Input				Output
A	B	C		Q
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		



7411 Triple 3 Input AND

Análise Circuitos: Exemplos:



3 input AND				
Input				Output
A	B	C		Q
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	1	0
1	1	1	1	1

Equação ?

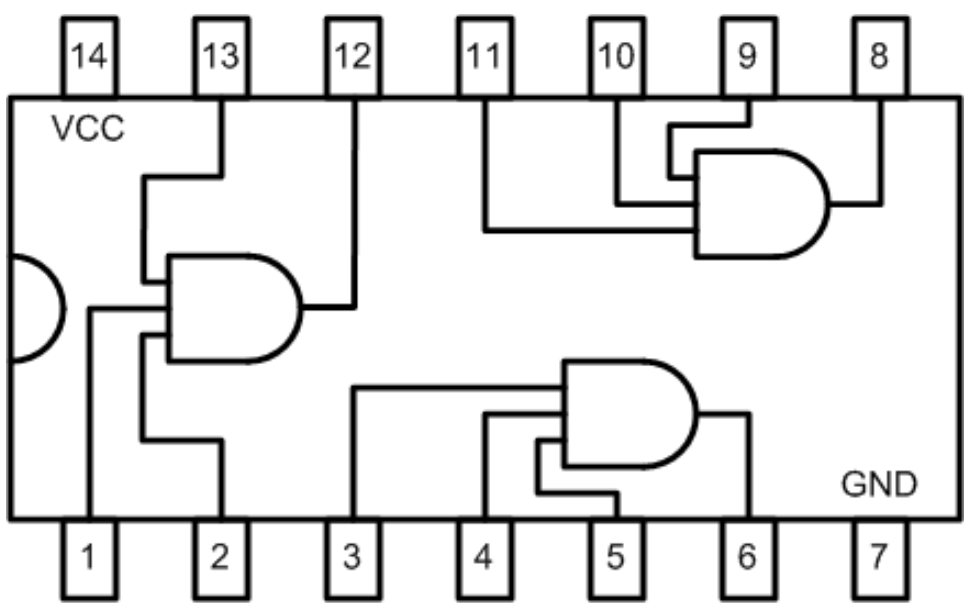
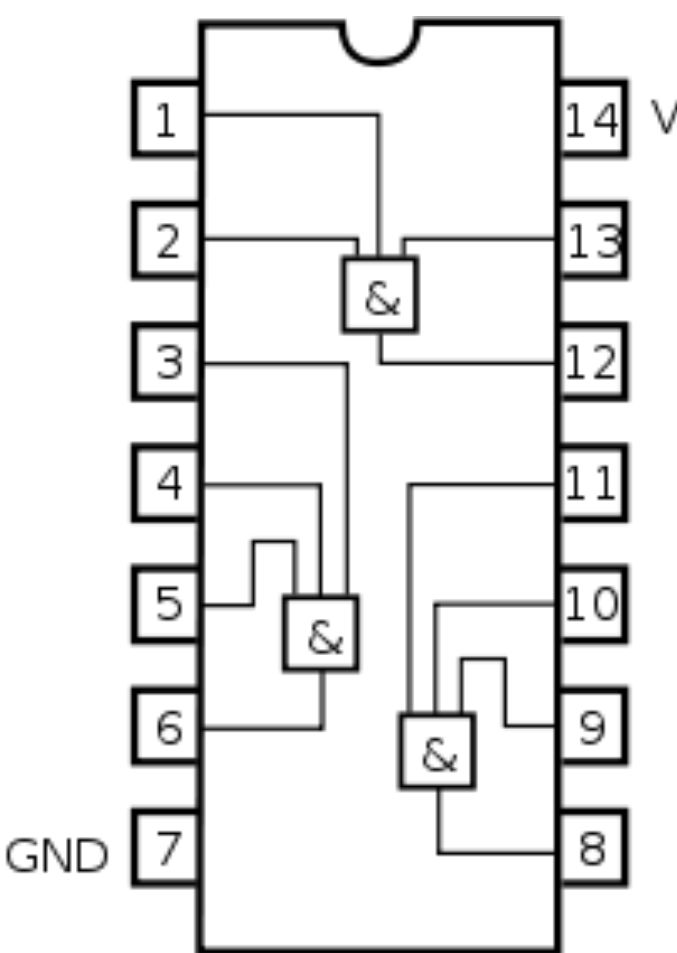
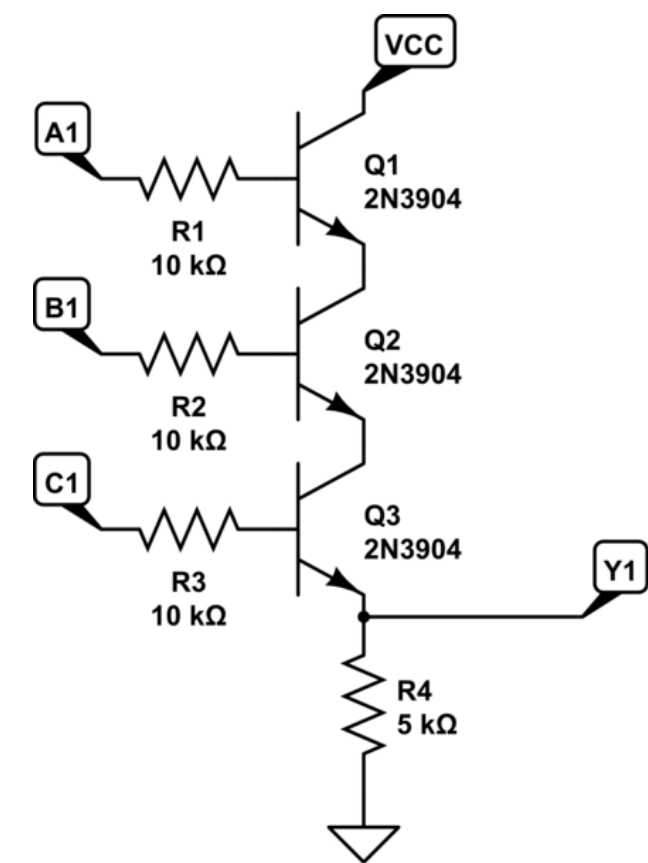
$$Q = A \cdot B \cdot C$$

Note:

Se $A = 0 \therefore Q = 0 \cdot B \cdot C = 0$

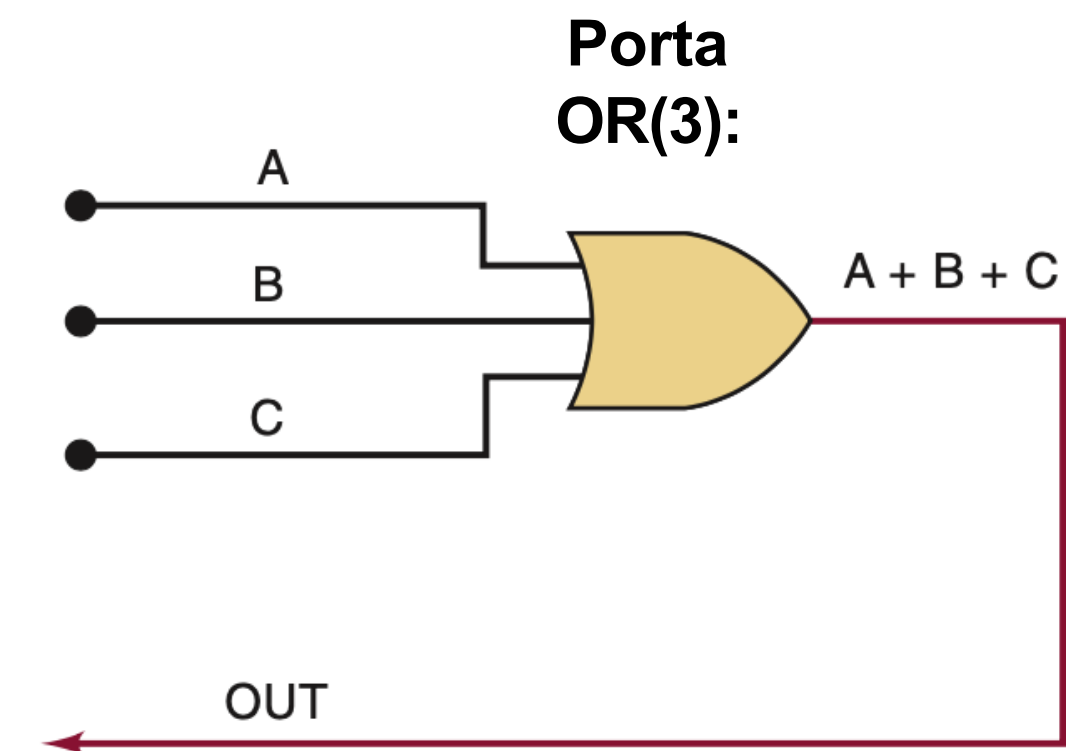
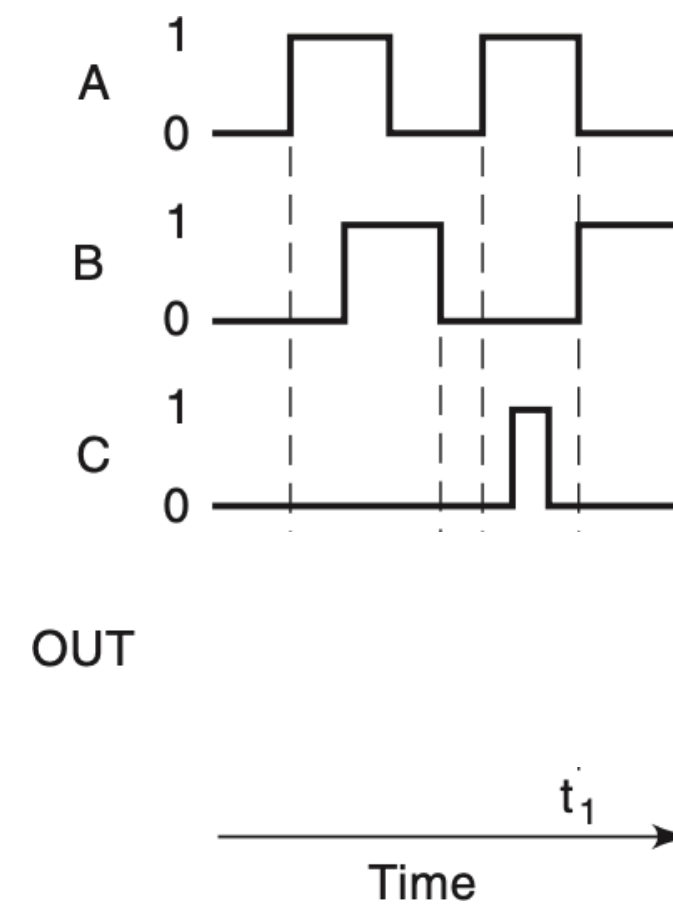
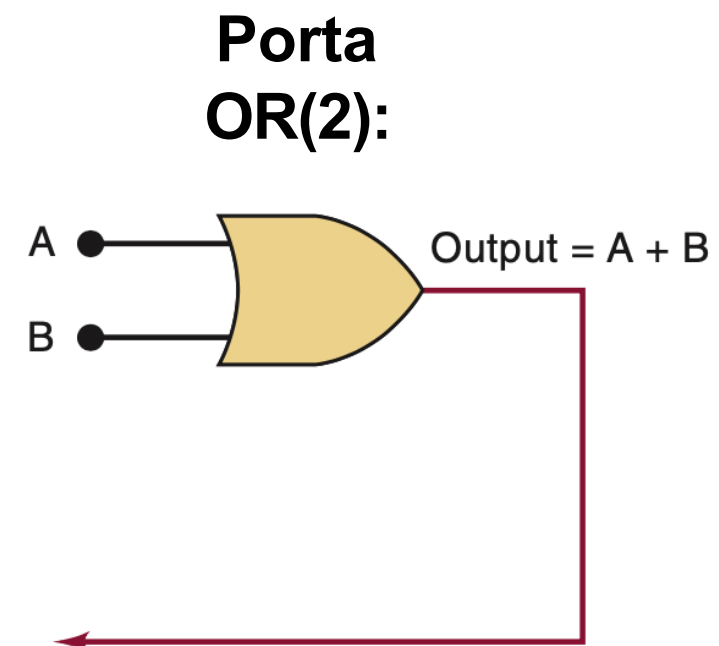
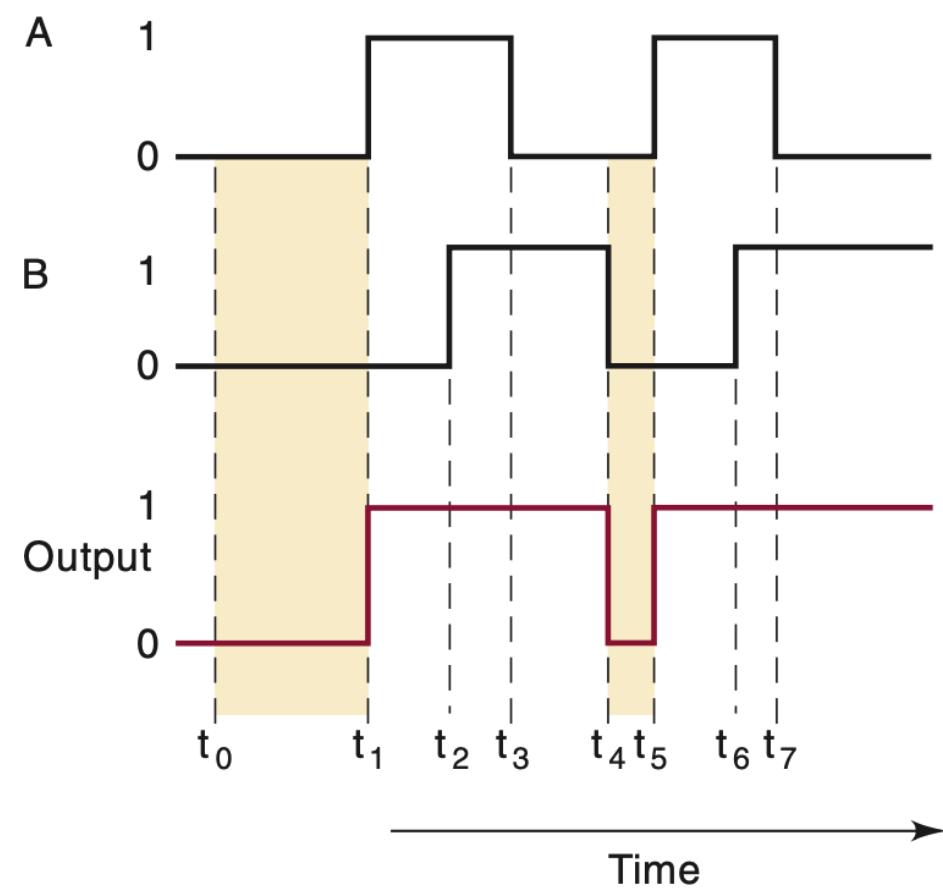
e

$$Q = 1 \iff A = 1, B = 1, C = 1$$
$$\Rightarrow Q = 1 \cdot 1 \cdot 1 = 1$$

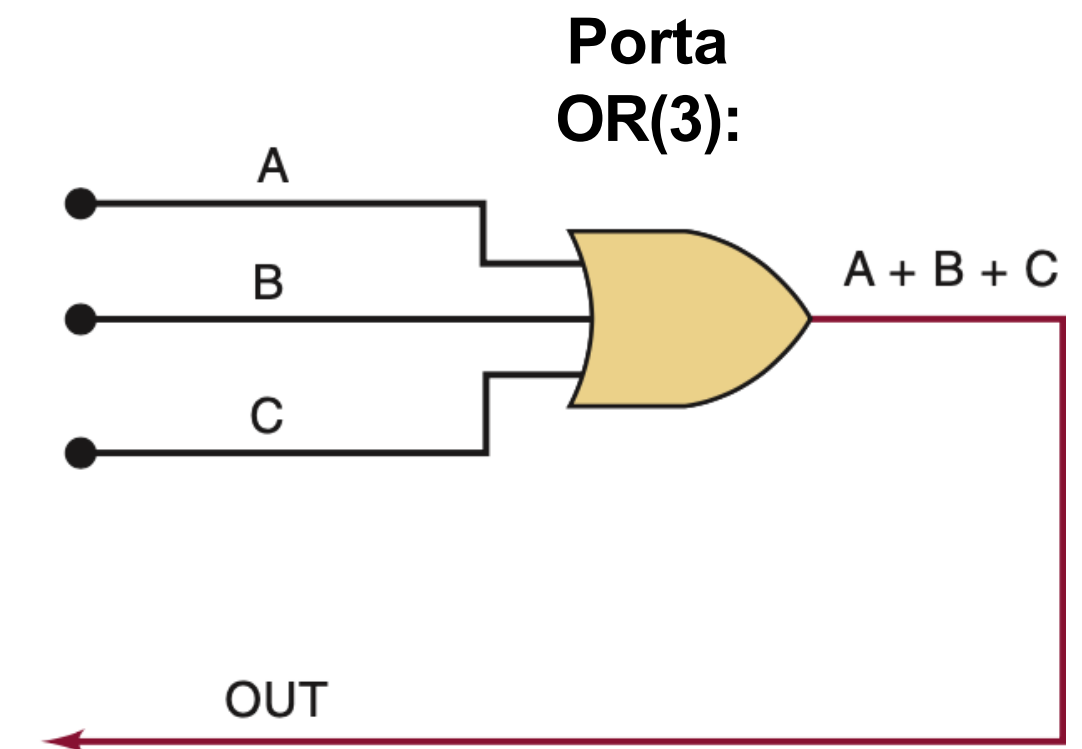
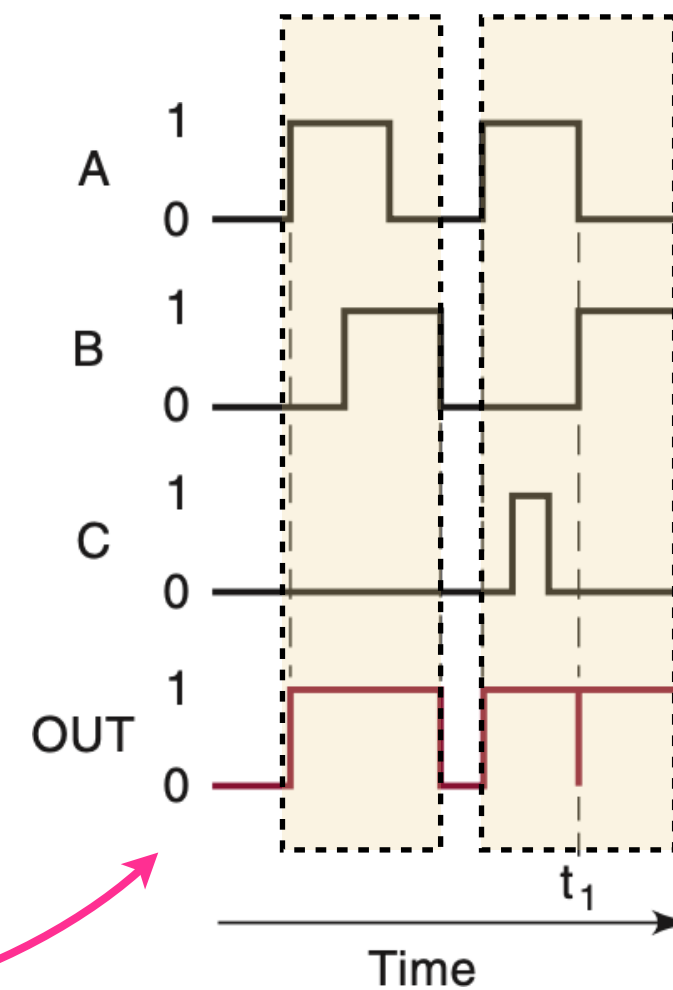
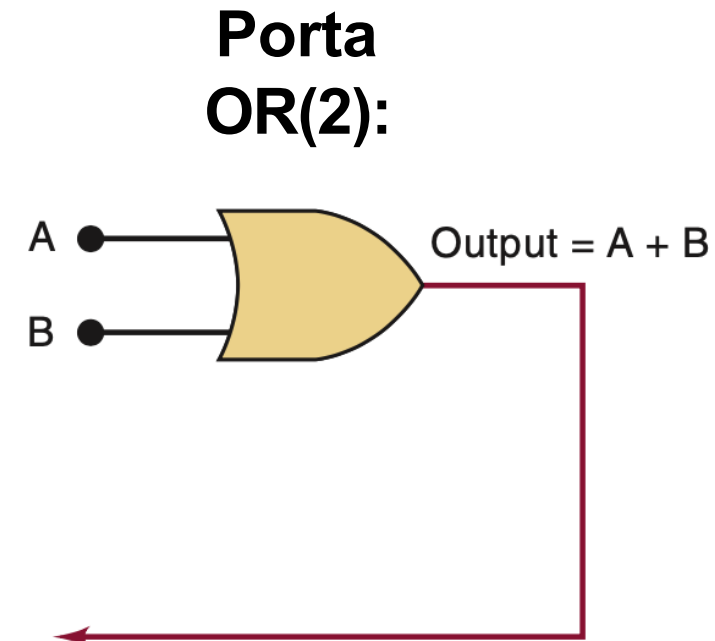
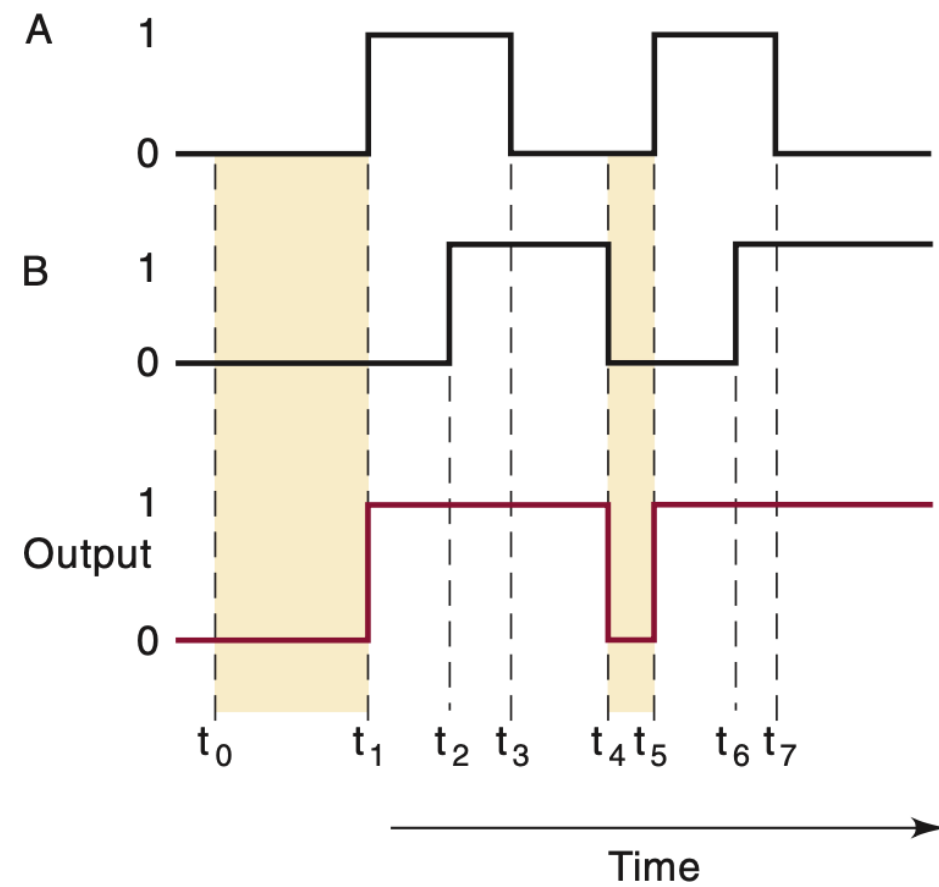


7411 Triple 3 Input AND

Análise Circuitos: Exemplos:



Análise Circuitos: Exemplos:



Porta OR:

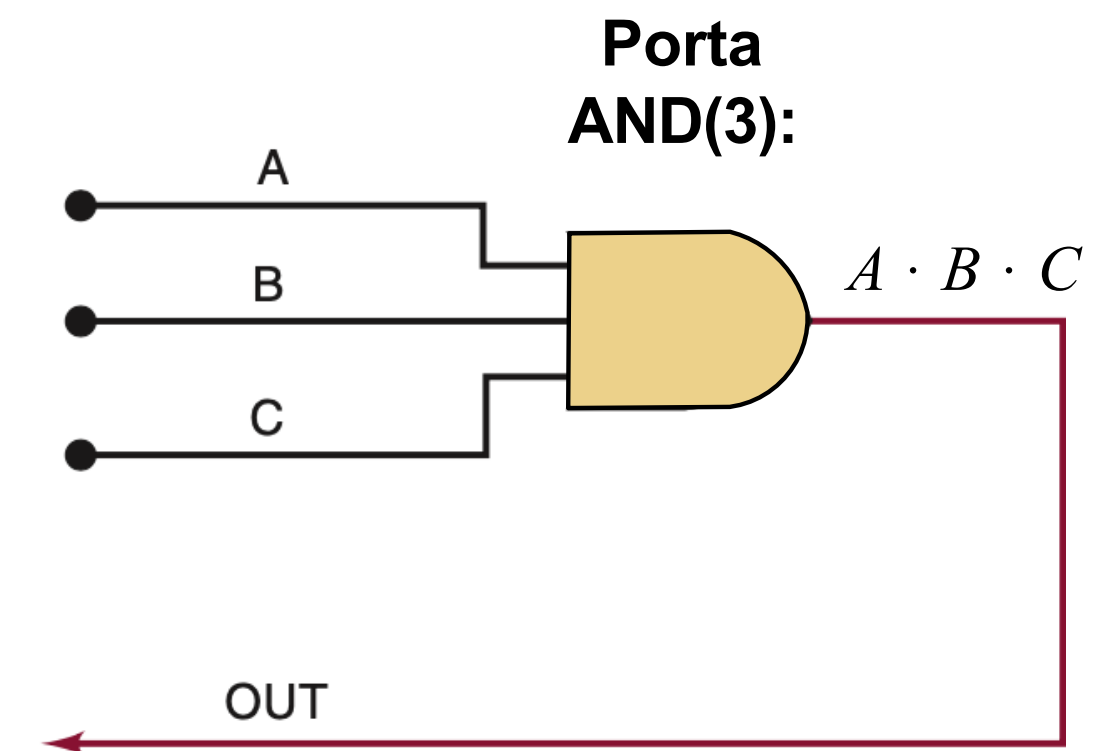
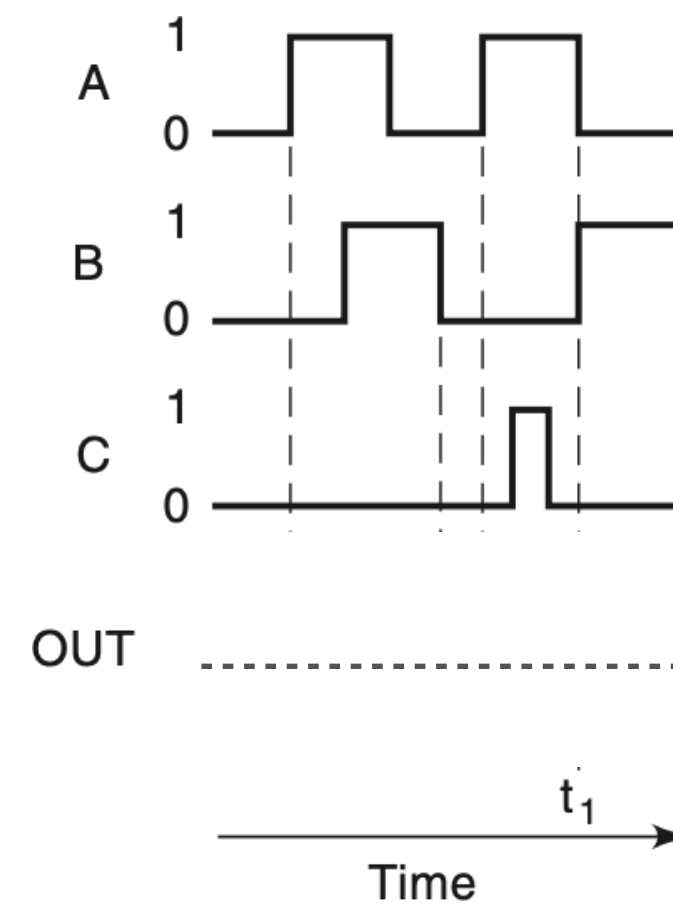
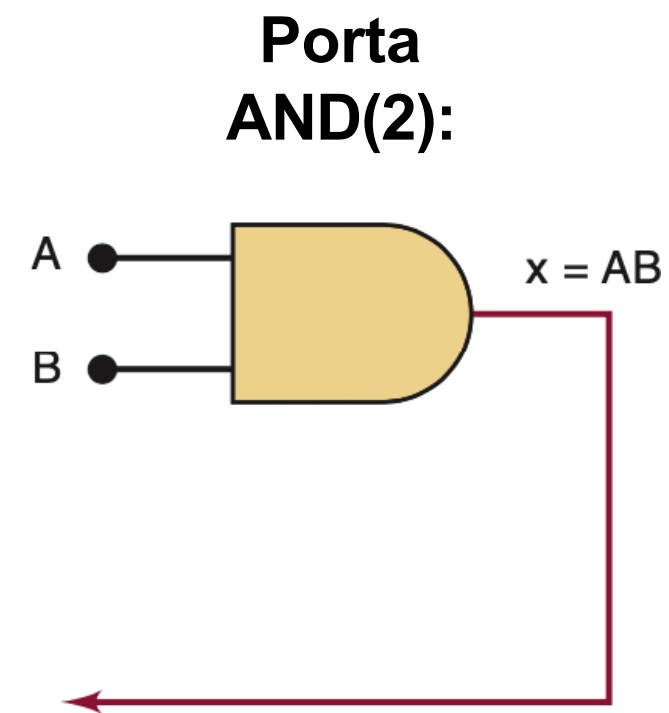
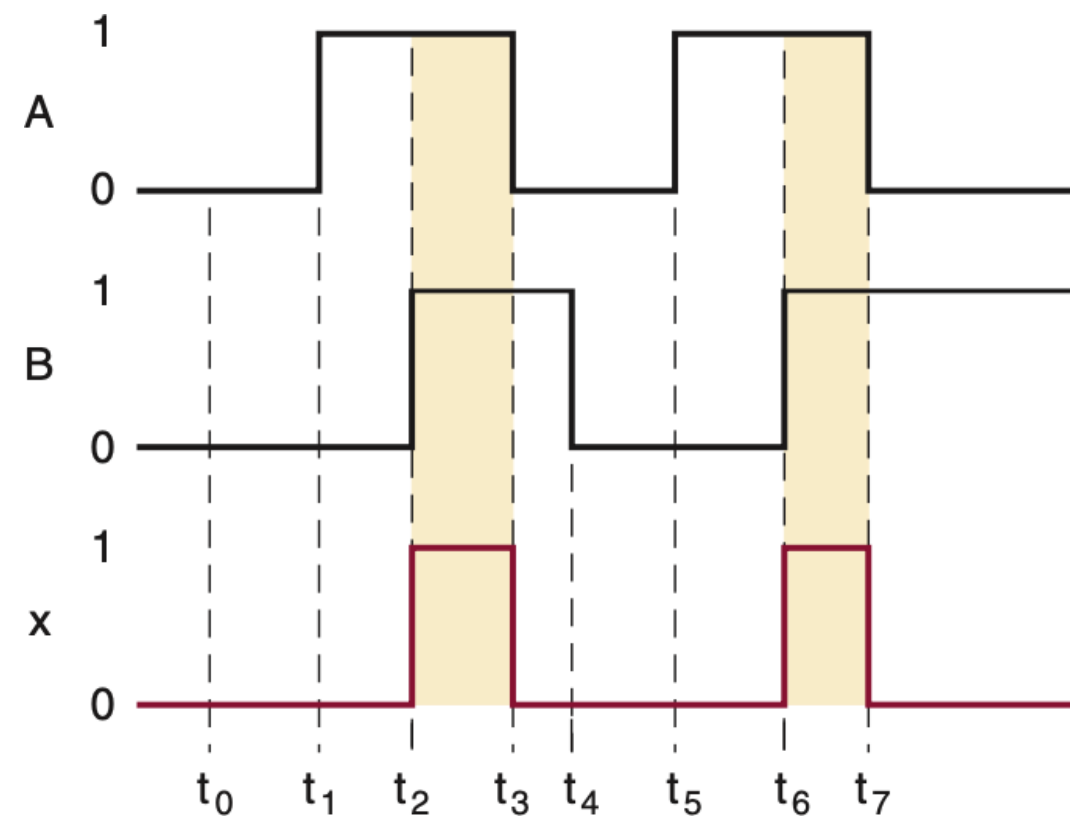
Note: basta uma entrada em nível lógico ALTO para saída comutar para nível lógico ALTO:

Álgebra de Boole:

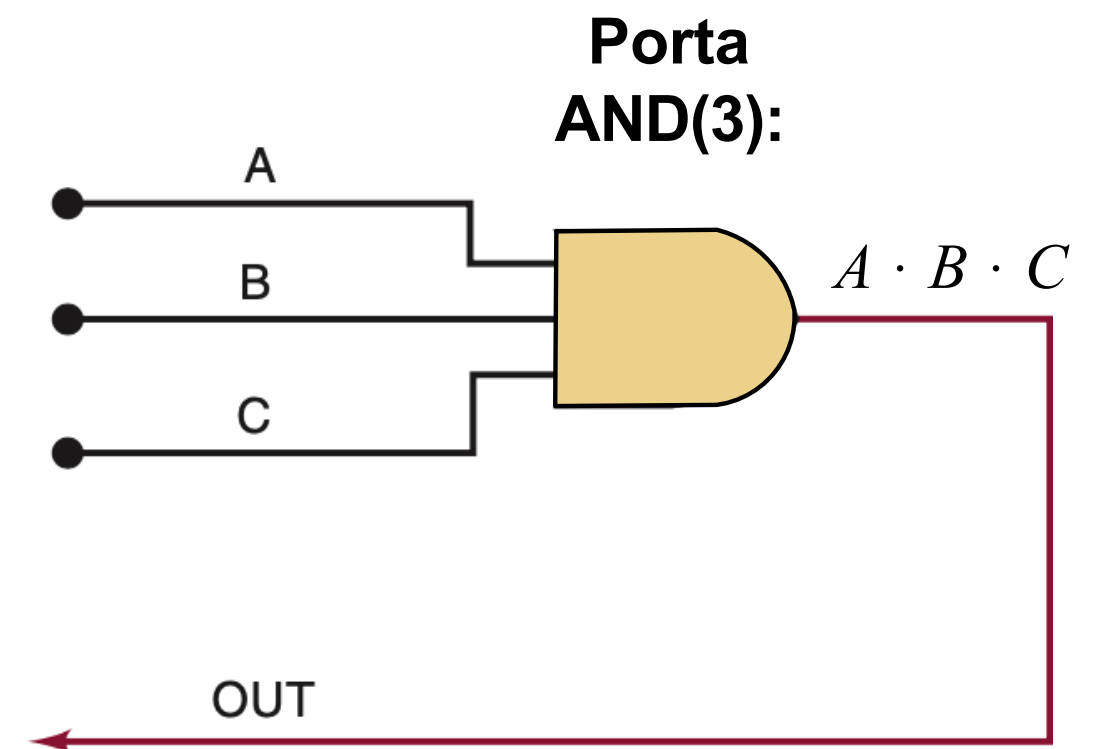
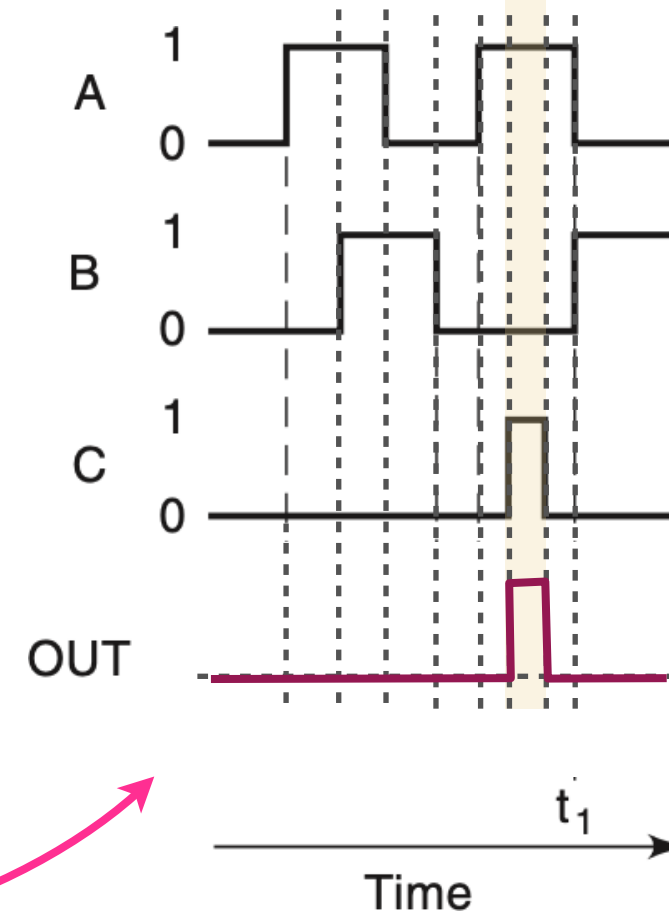
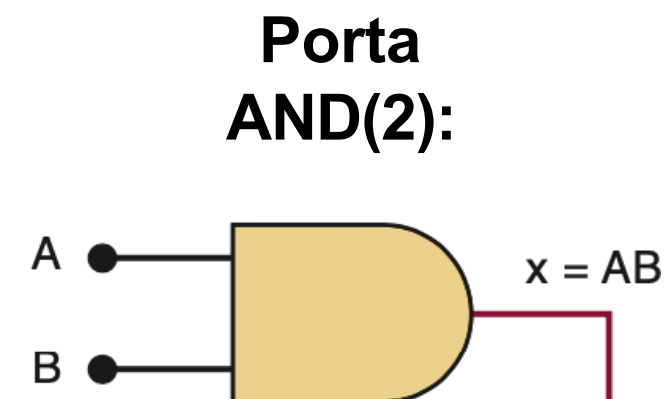
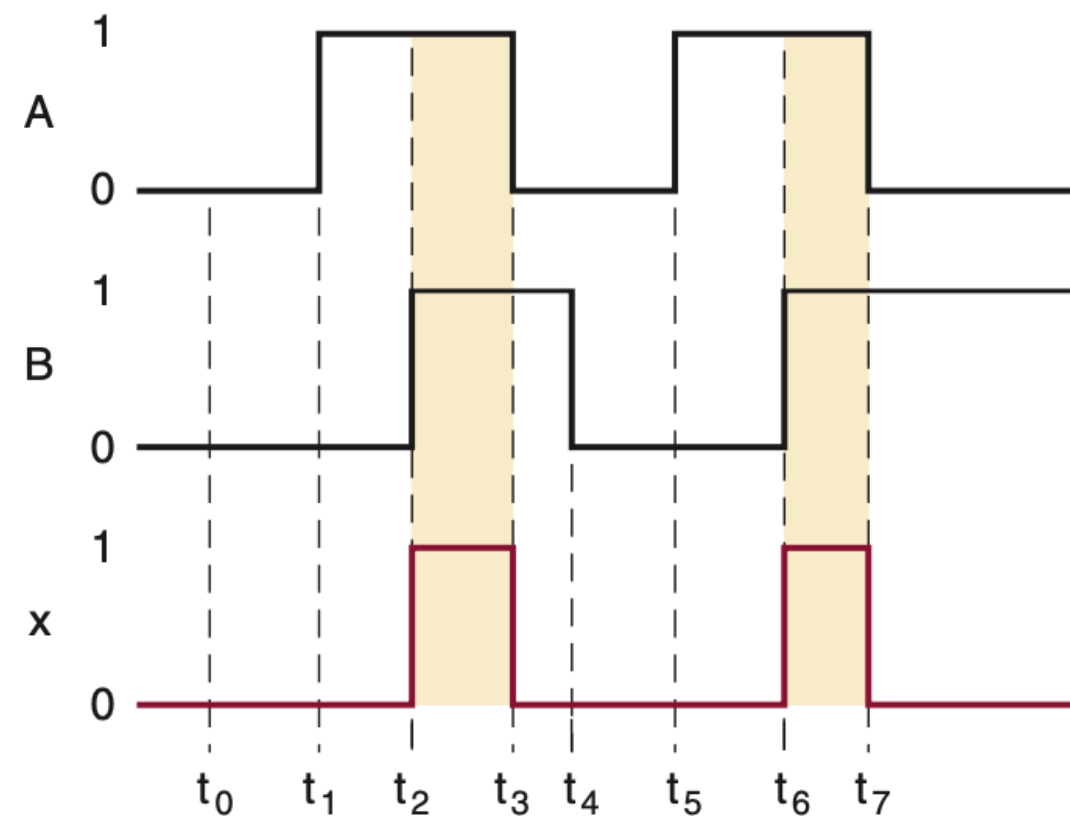
$$x + 0 = x$$

$$x + 1 = 1$$

Análise Circuitos: Exemplos:



Análise Circuitos: Exemplos:



Porta AND:

Note: basta uma entrada em nível lógico BAIXO para saída comutar para nível lógico BAIXO

Álgebra de Boole:

$$x \cdot 0 = 0$$

$$x \cdot 1 = x$$

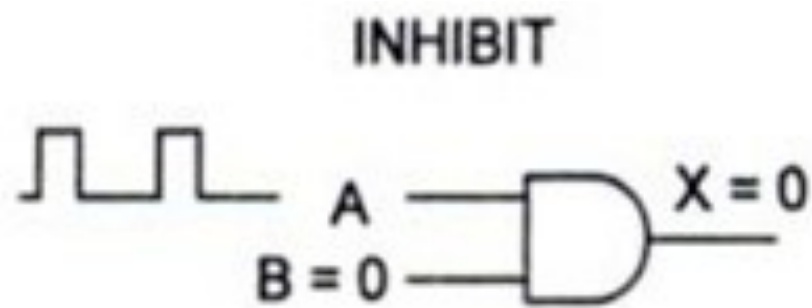
Intro à Álgebra de Boole:

Porta AND:



Note:

$$x \cdot 1 = x$$

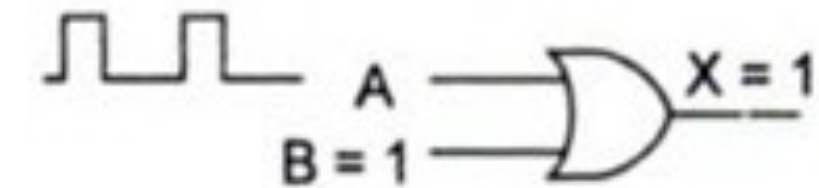


$$x \cdot 0 = 0$$

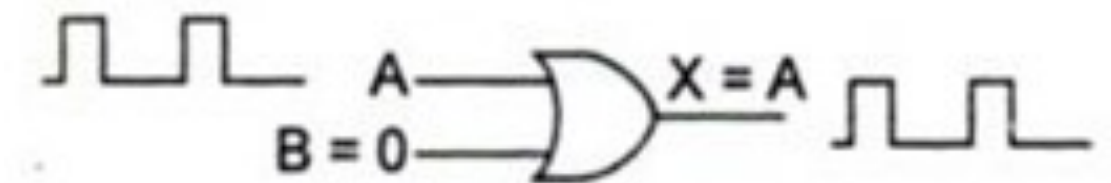
Porta OR:

Note:

$$x + 1 = 1$$



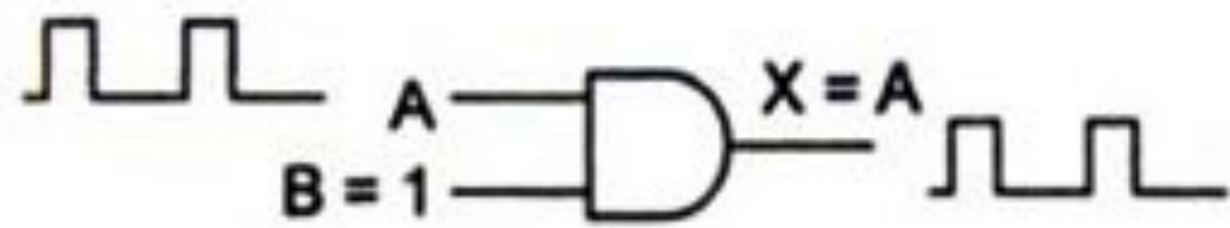
$$x + 0 = x$$



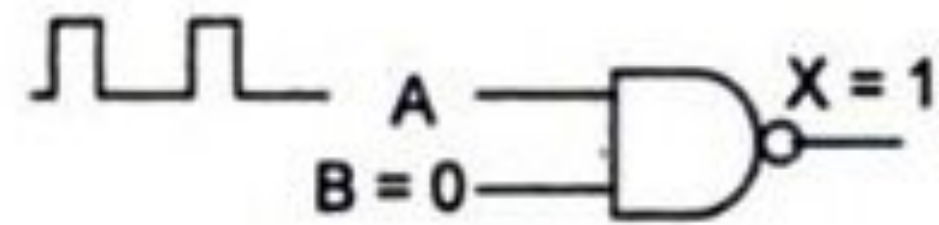
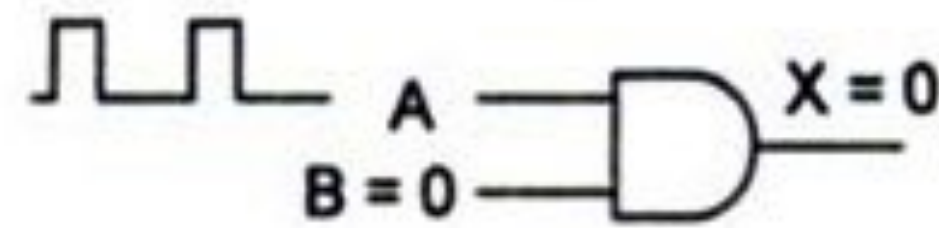
Obs.: A porta AND pode funcionar como uma “chave” com Entrada de “Enable”

Entrada Enable

ENABLE



INHIBIT

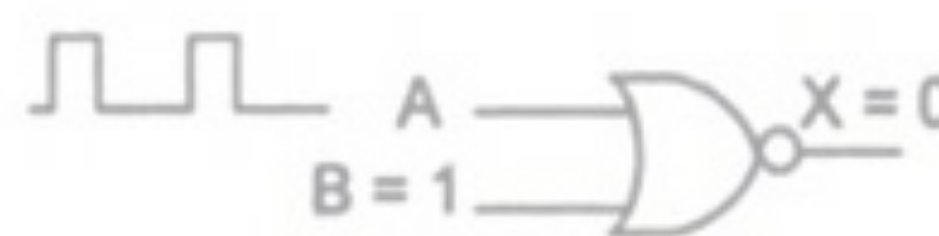
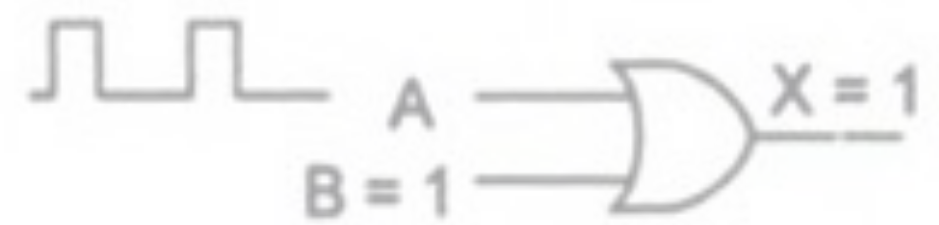


Porta AND:

Note:

$$x \cdot 0 = 0$$

$$x \cdot 1 = x$$



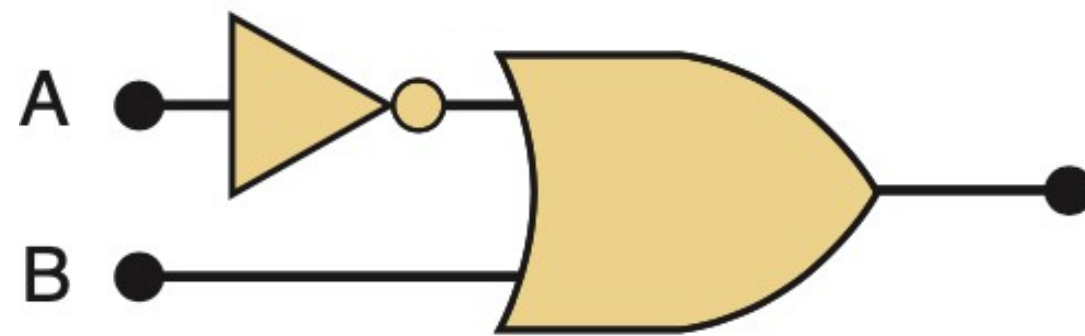
Porta OR:

Note:

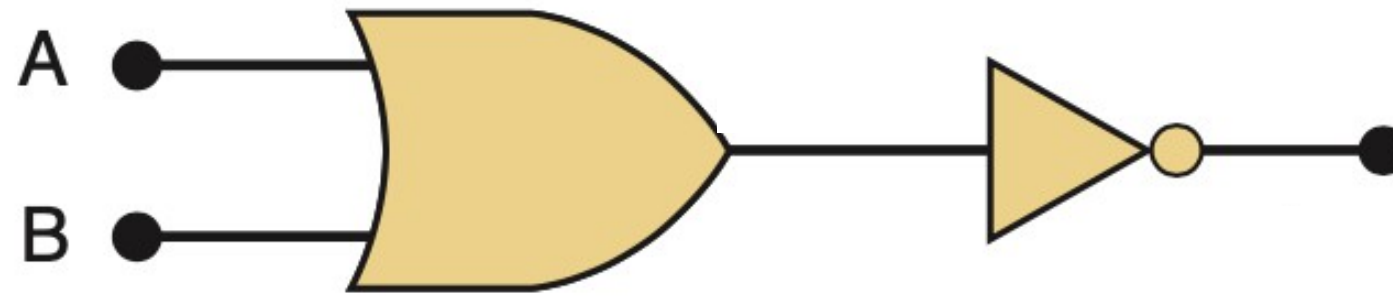
$$x + 0 = x$$

$$x + 1 = 1$$

Análise Circuitos: Exemplos:



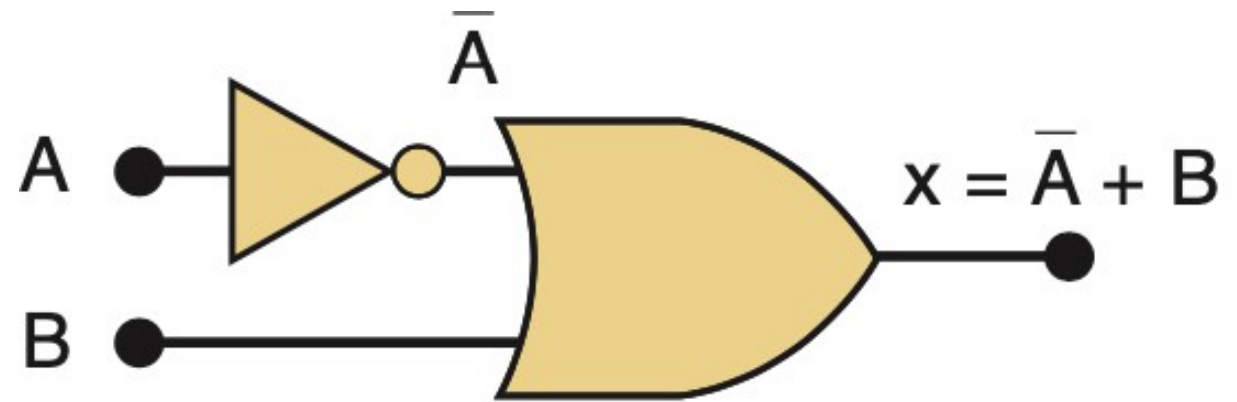
(a)



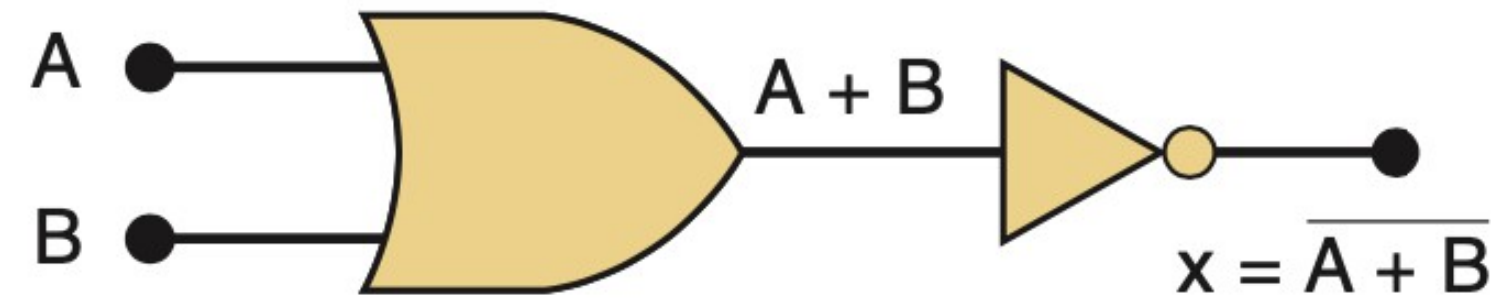
(b)

A B	(a)	A B	(b)
0 0		0 0	
0 1		0 1	
1 0		1 0	
1 1		1 1	

Análise Circuitos: Exemplos:



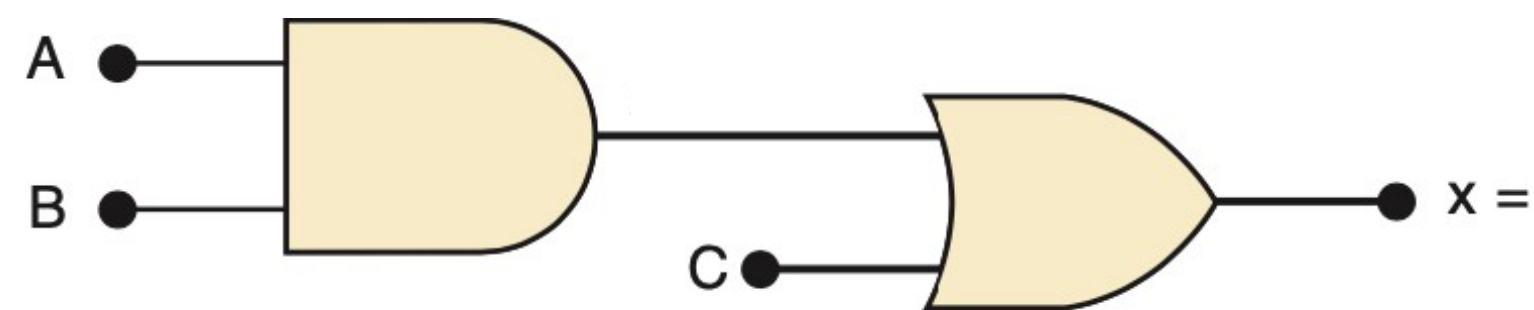
(a)



(b)

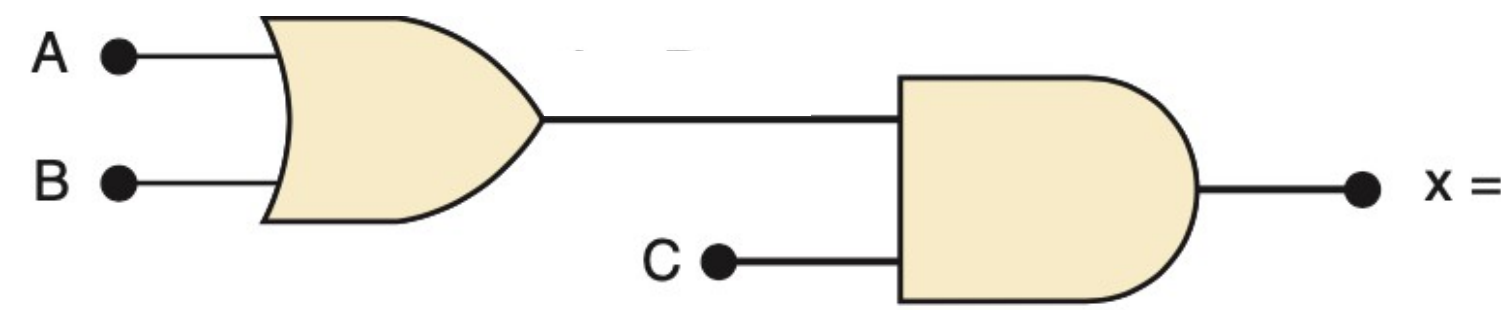
A B	(a)	A B	(b)
0 0		0 0	
0 1		0 1	
1 0		1 0	
1 1		1 1	

Análise Circuitos: Exemplos:



(a)

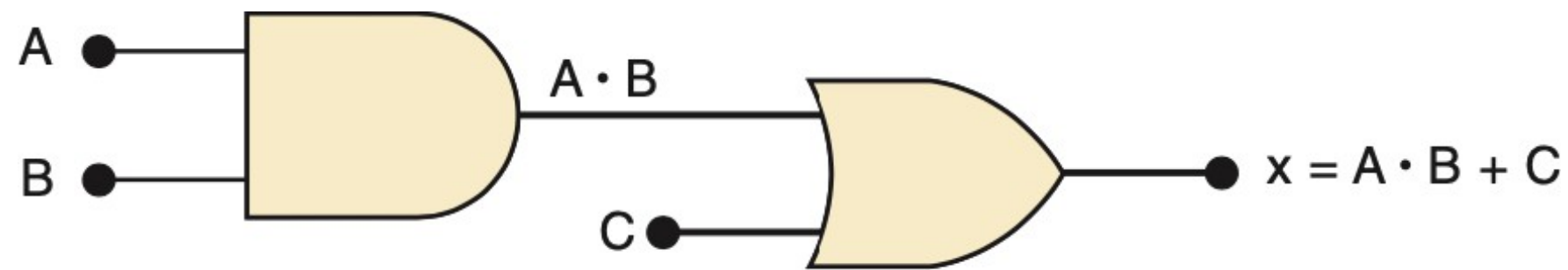
Ref	A	B	C	$A \cdot B$	x
0					
1					
2					
3					
4					
5					
6					
7					



(b)

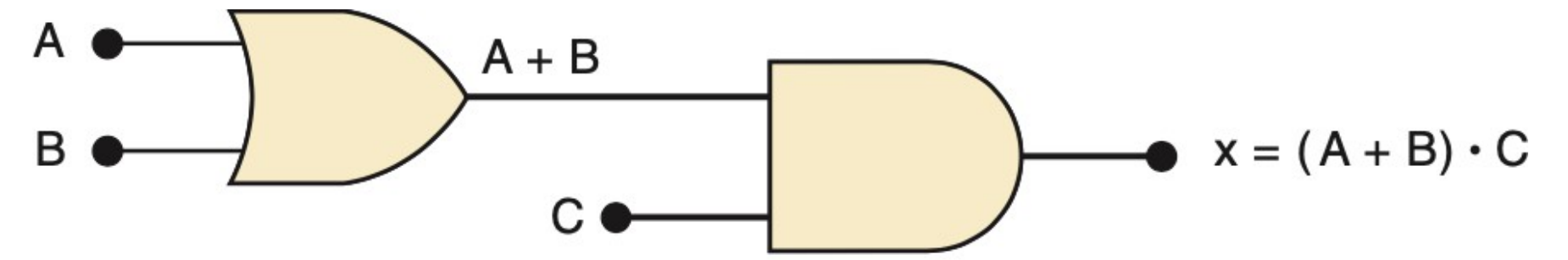
Ref	A	B	C	$A + B$	x
0					
1					
2					
3					
4					
5					
6					
7					

Análise Circuitos: Exemplos:



(a)

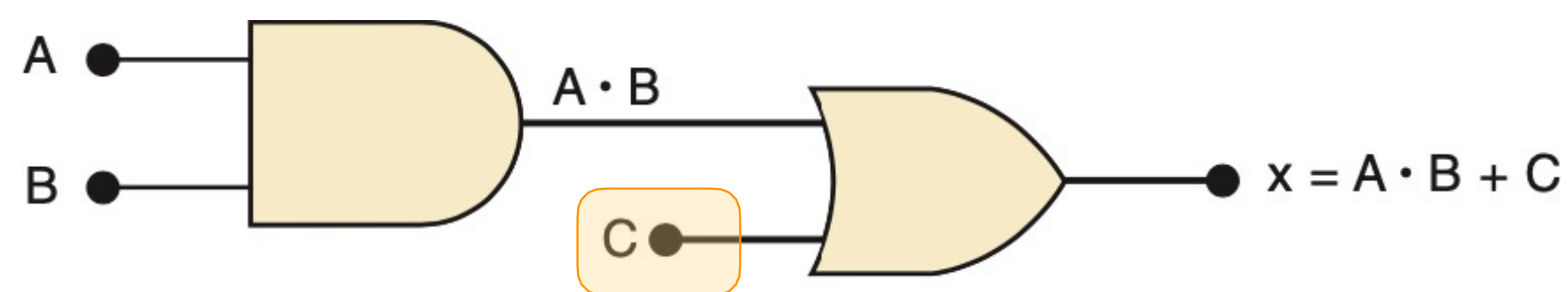
Ref	A	B	C	$A \cdot B$	x
0	0	0	0		
1	0	0	1		
2	0	1	0		
3	0	1	1		
4	1	0	0		
5	1	0	1		
6	1	1	0		
7	1	1	1		



(b)

Ref	A	B	C	$A + B$	x
0	0	0	0		
1	0	0	1		
2	0	1	0		
3	0	1	1		
4	1	0	0		
5	1	0	1		
6	1	1	0		
7	1	1	1		

Análise Circuitos: Exemplos:

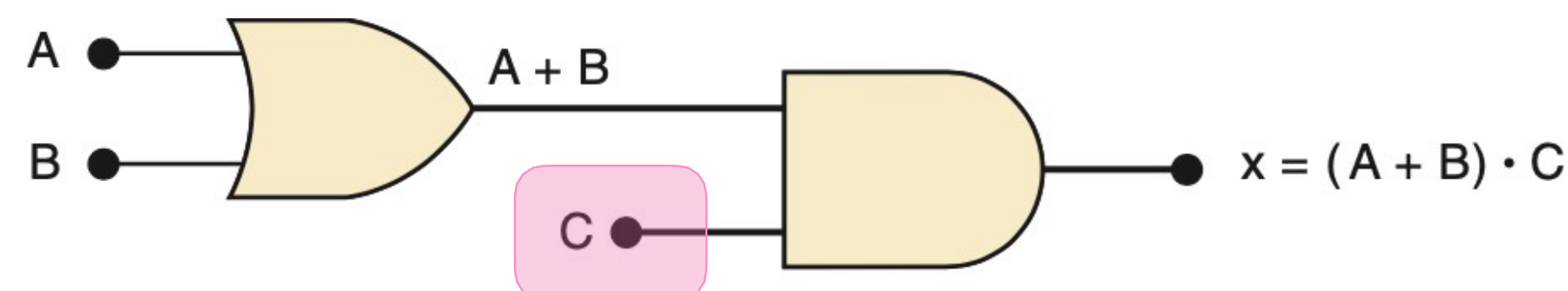


(a)

Ref	A	B	C	$A \cdot B$	x
0	0	0	0		
1	0	0	1		1
2	0	1	0		
3	0	1	1		1
4	1	0	0		
5	1	0	1		1
6	1	1	0		
7	1	1	1		1

$$x + 1 = 1$$

Porta OR:
Basta uma
entrada = "1",
Saída = "1"



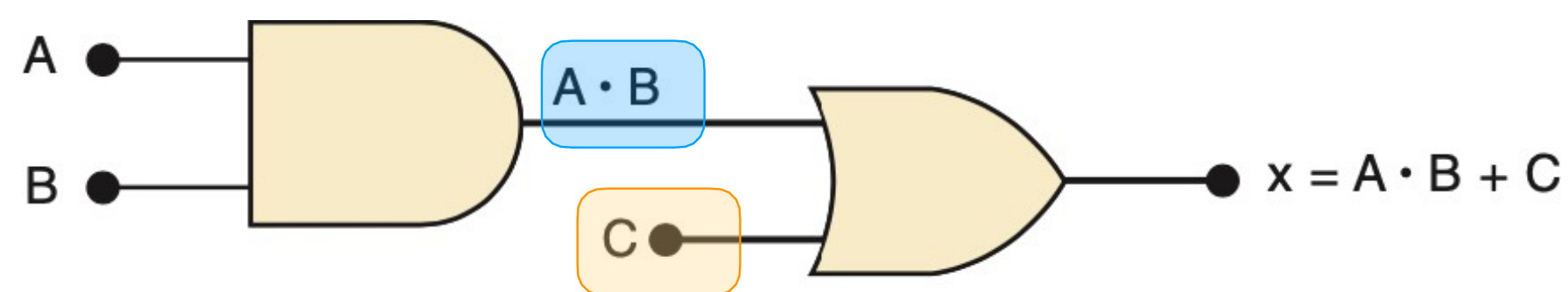
(b)

Ref	A	B	C	$A + B$	x
0	0	0	0		0
1	0	0	1		
2	0	1	0		0
3	0	1	1		
4	1	0	0		0
5	1	0	1		
6	1	1	0		0
7	1	1	1		

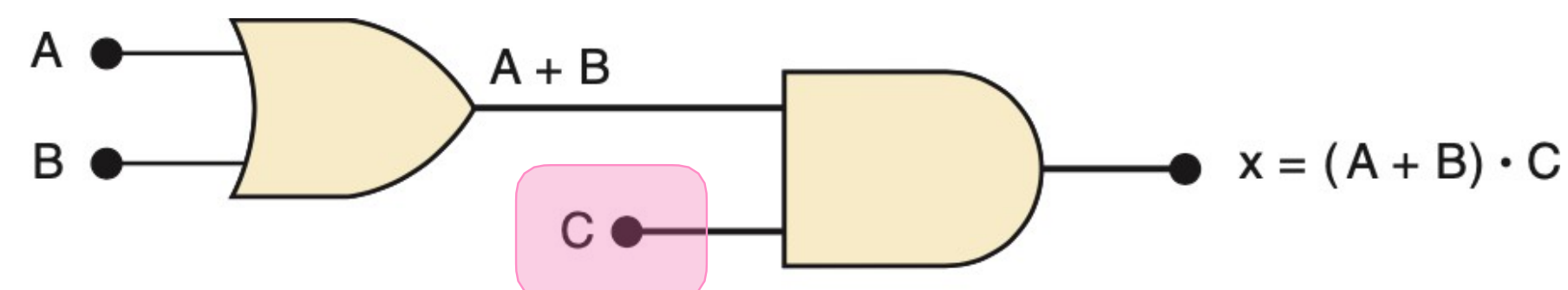
$$x \cdot 0 = 0$$

Porta AND:
Basta uma
entrada = "0",
Saída = "0"

Análise Circuitos: Exemplos:



(a)



(b)

Ref	A	B	C	$A \cdot B$	x
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	0
3	0	1	1	0	1
4	1	0	0	0	0
5	1	0	1	0	1
6	1	1	0	1	1
7	1	1	1	1	1

$$x + 1 = 1$$

Porta OR:
Basta uma
entrada = "1",
Saída = "1"

Saída da AND = "1" apenas
Quando TODAS as entradas = "1".

$$1 \cdot 1 \Leftrightarrow 1$$

Ref	A	B	C	$A + B$	x
0	0	0	0	0	0
1	0	0	1	0	0
2	0	1	0	1	0
3	0	1	1	1	1
4	1	0	0	1	0
5	1	0	1	1	1
6	1	1	0	1	0
7	1	1	1	1	1

$$x \cdot 0 = 0$$

Porta AND:
Basta uma
entrada = "0",
Saída = "0"

Porta OR:
Basta uma entrada = "1",
Saída = "1"

Exemplo:

- Que porta é esta?

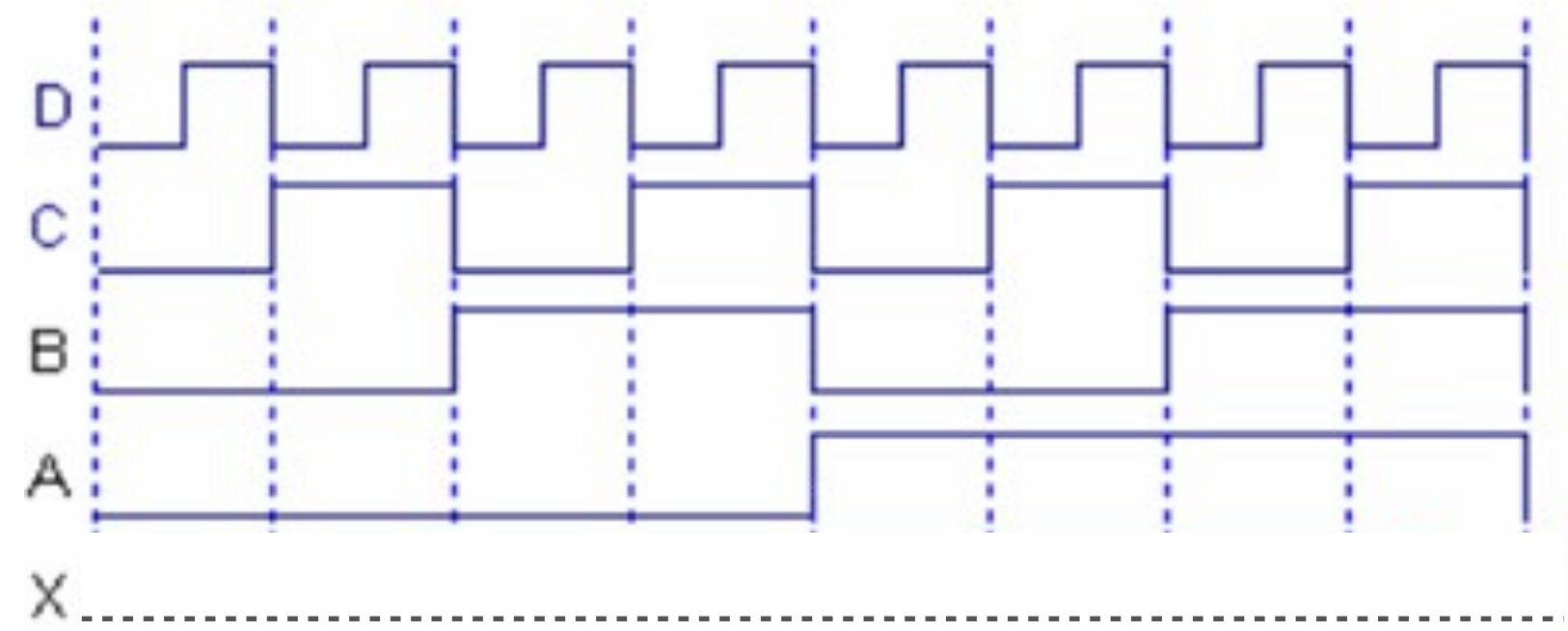
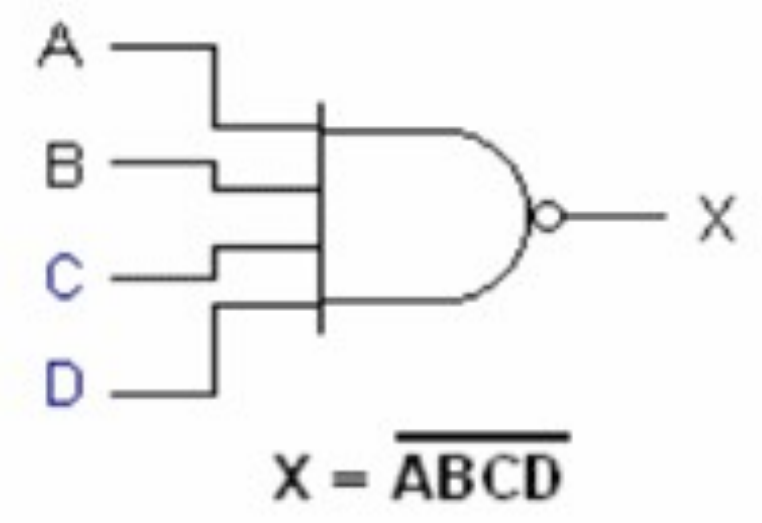
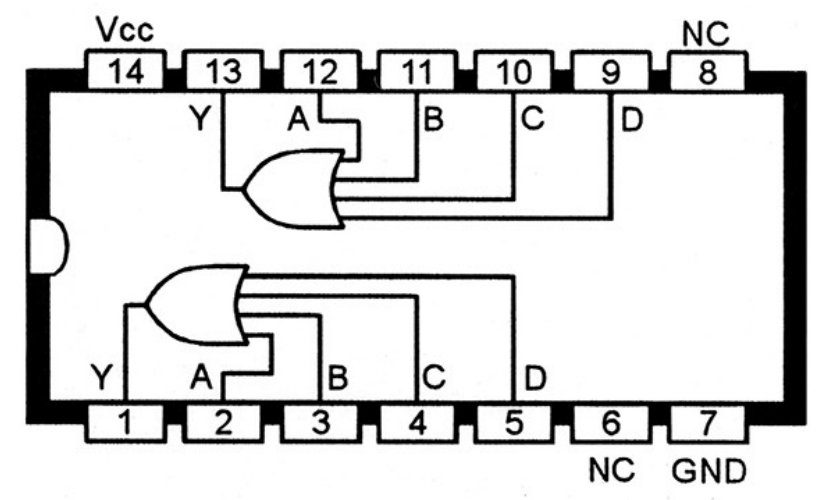


Diagrama de formas de onda

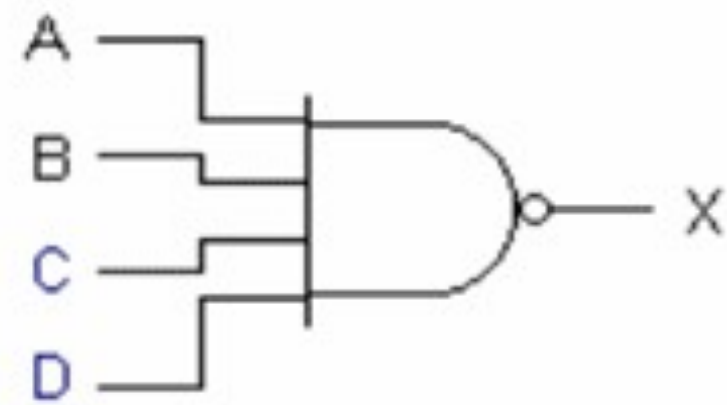
A	B	C	D	X
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	



Exemplo:

NAND(4)

- Que porta é esta?



$$X = \overline{ABCD}$$

Note:

$$x \cdot 0 = 0$$

$$x \cdot 1 = x$$

A	B	C	D	X
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

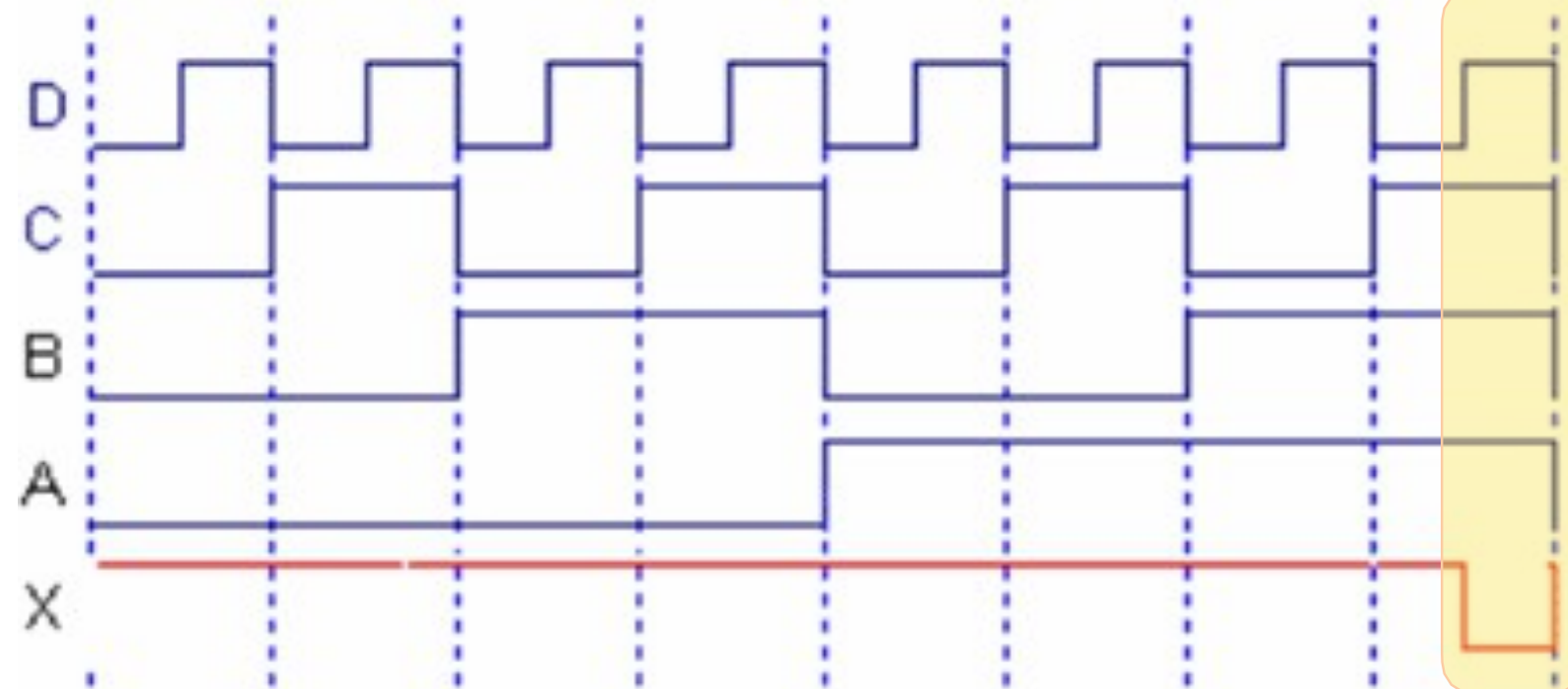
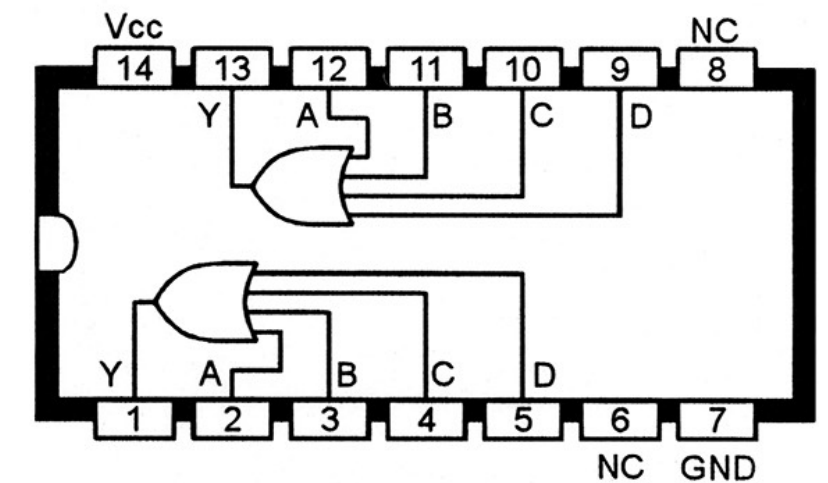
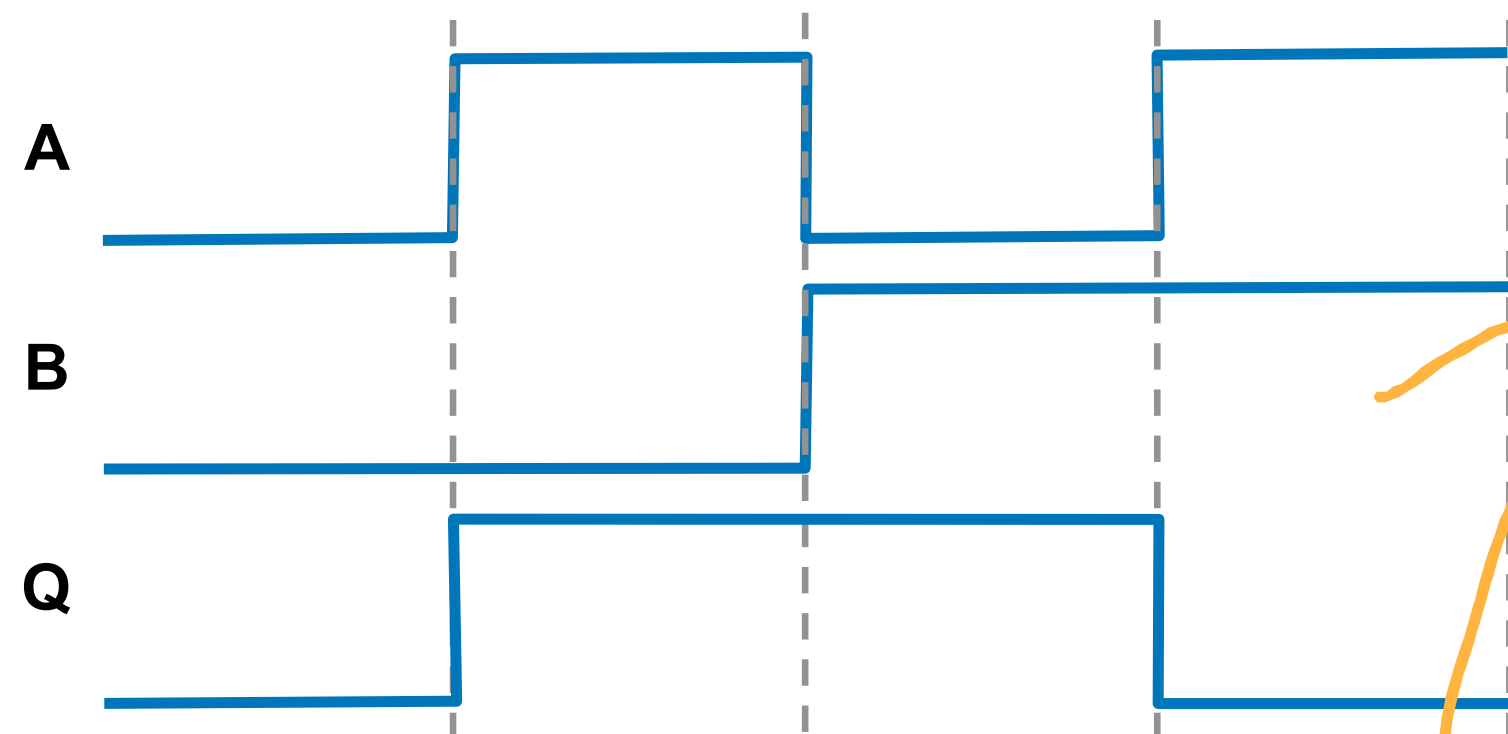
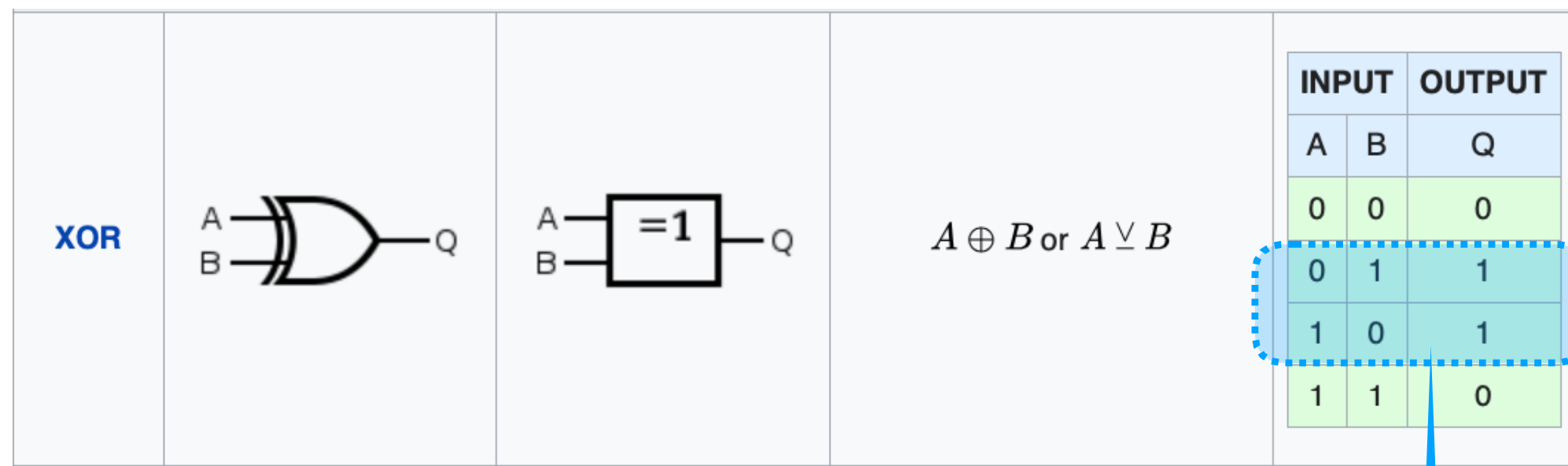


Diagrama de formas de onda



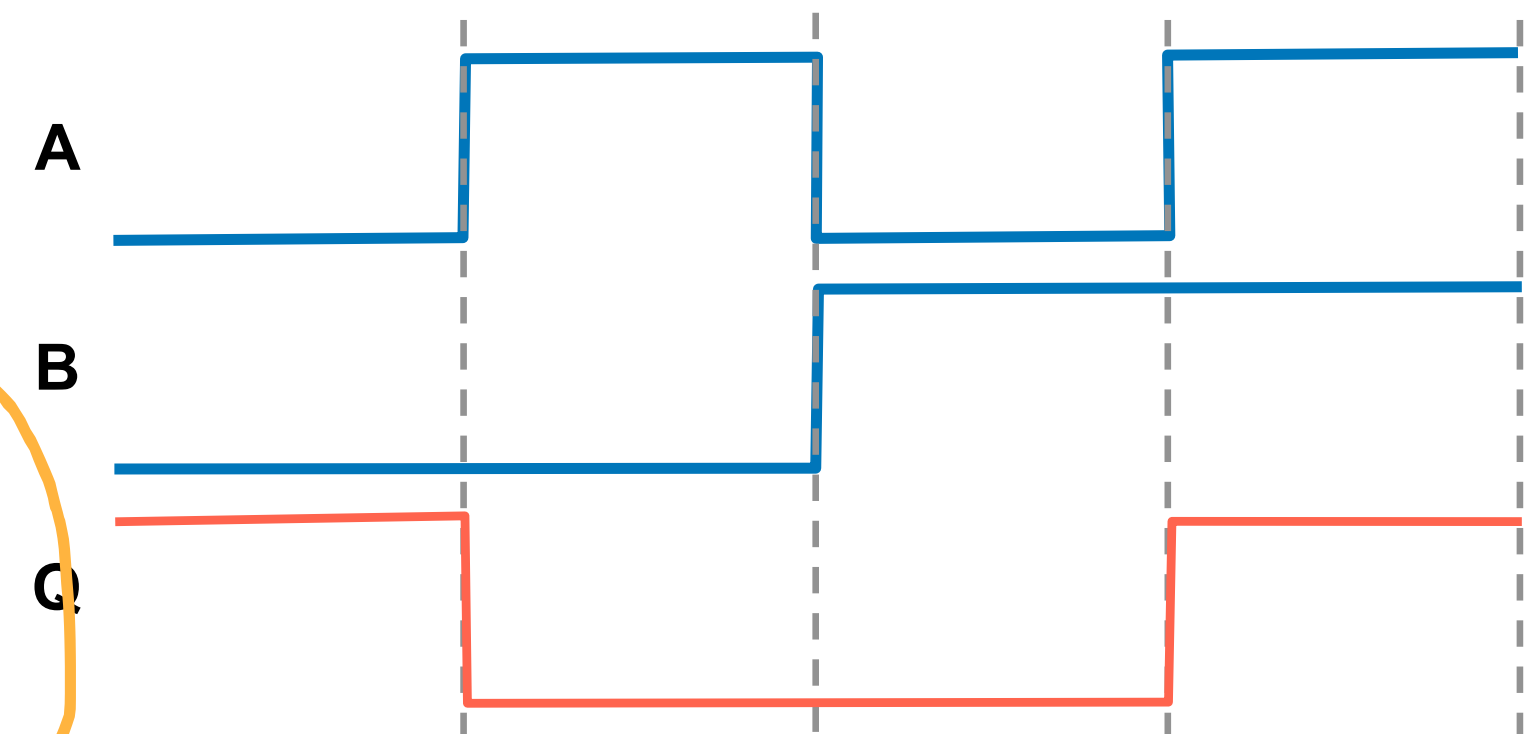
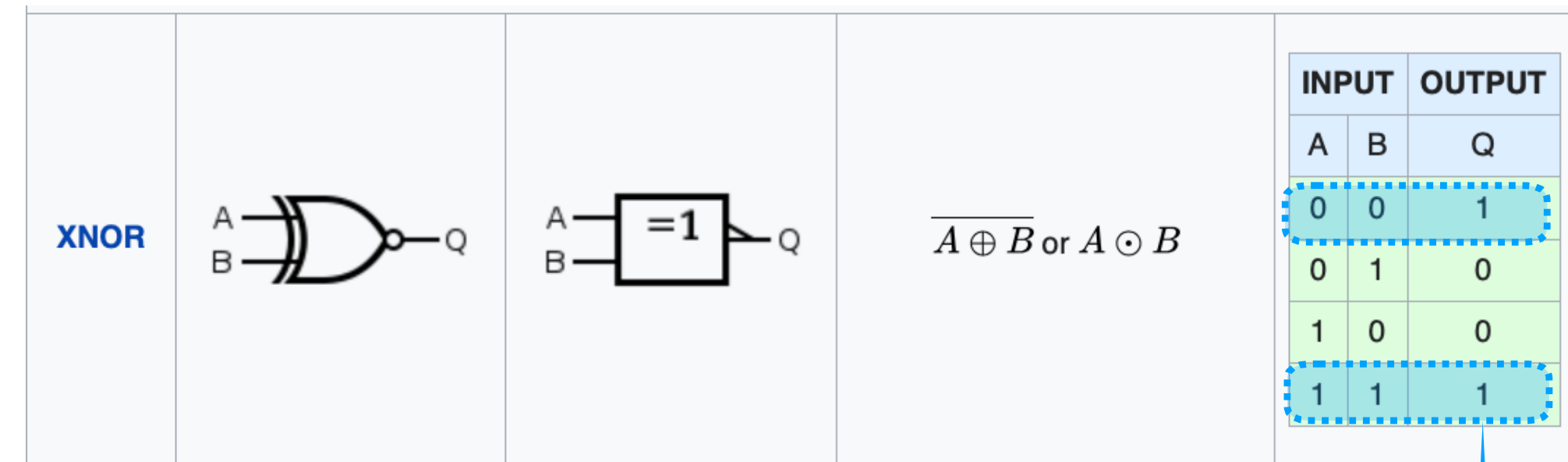
Últimas portas: XOR e NXOR

- XOR: eXclusive OR:



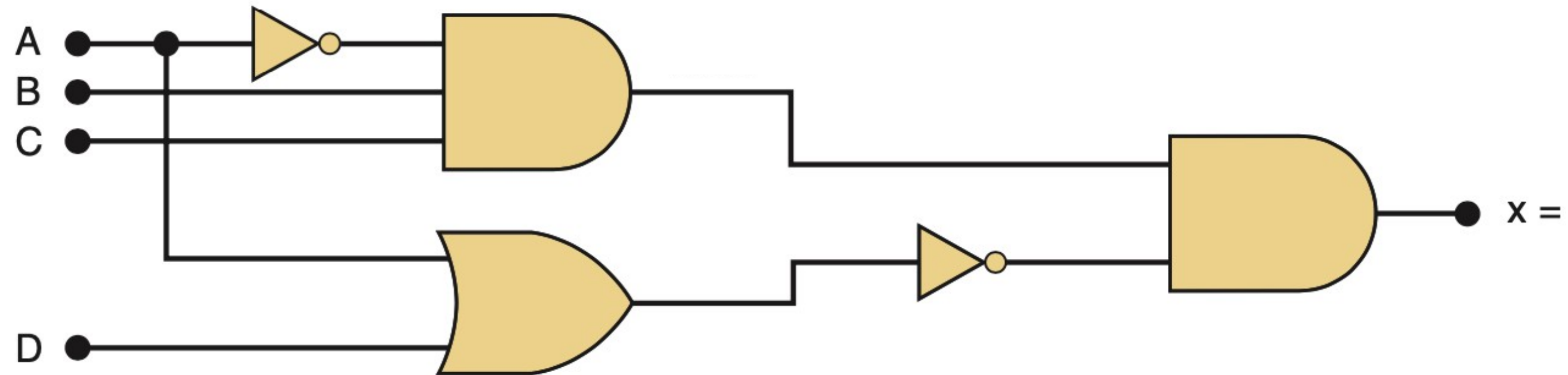
Detector de
Desigualdades

- N**XOR: Negative eXclusive OR:

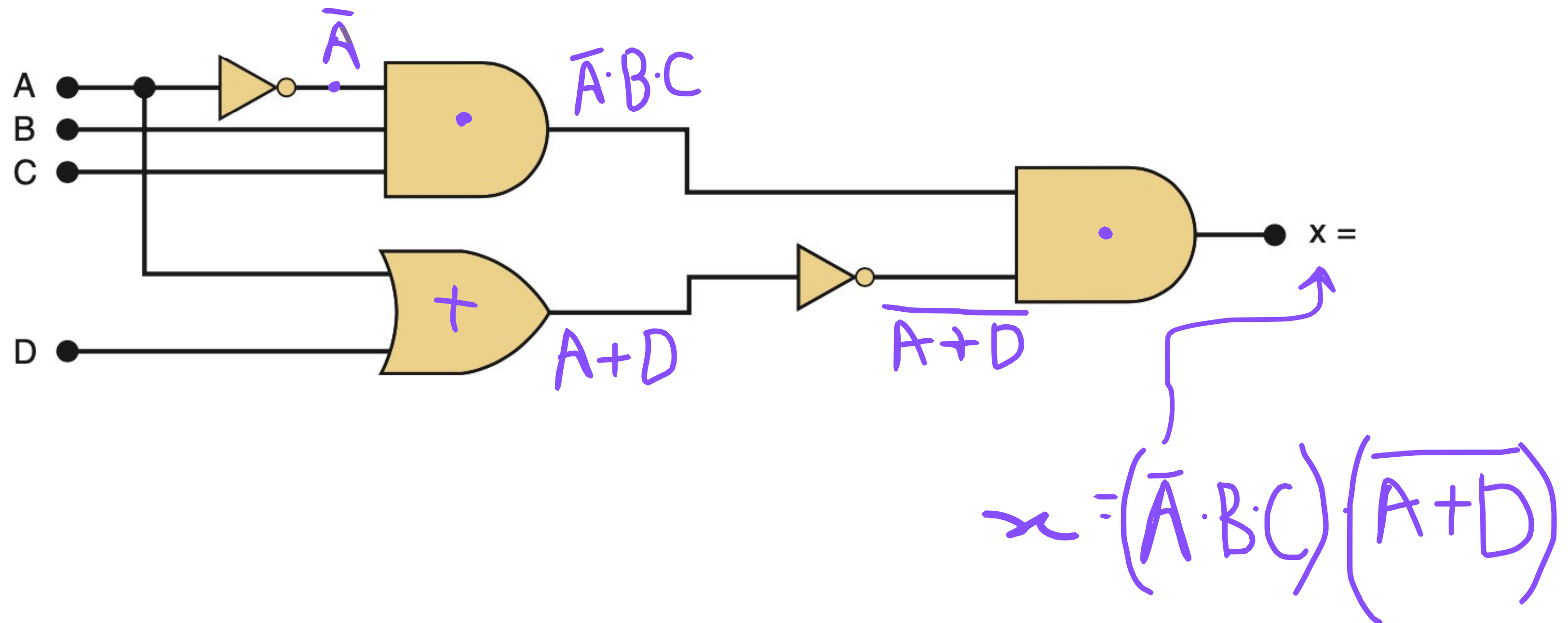


Detector de
Igualdades

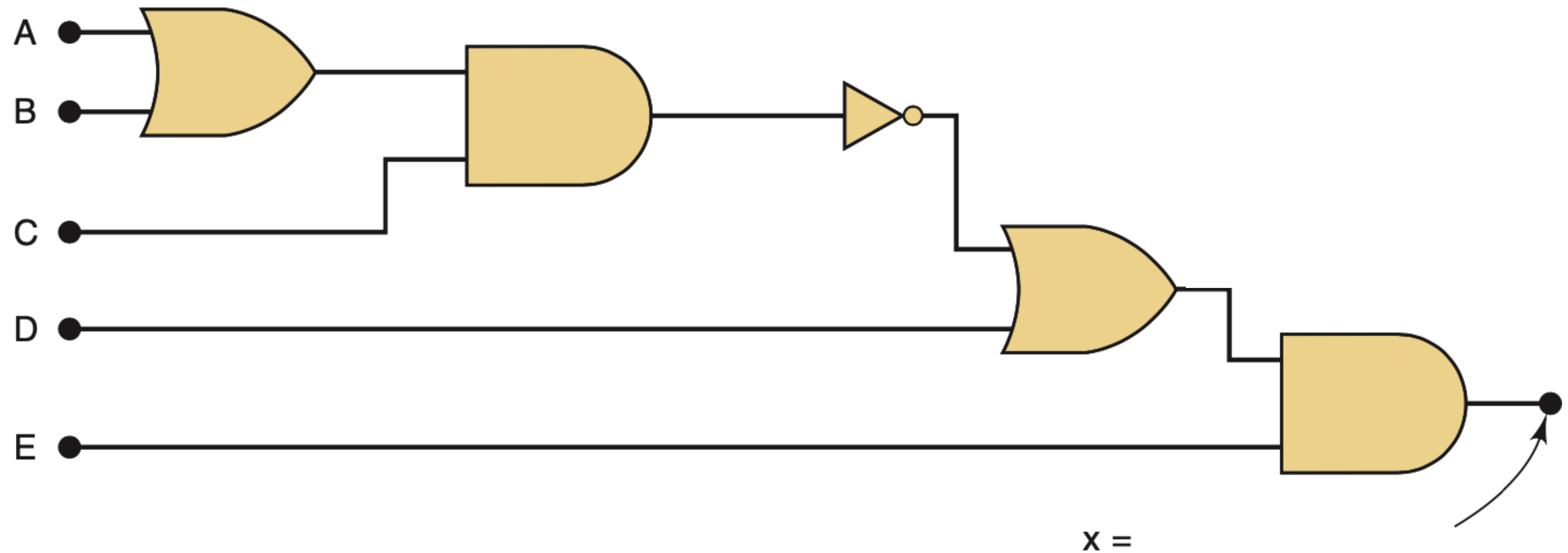
Deduzindo equações:



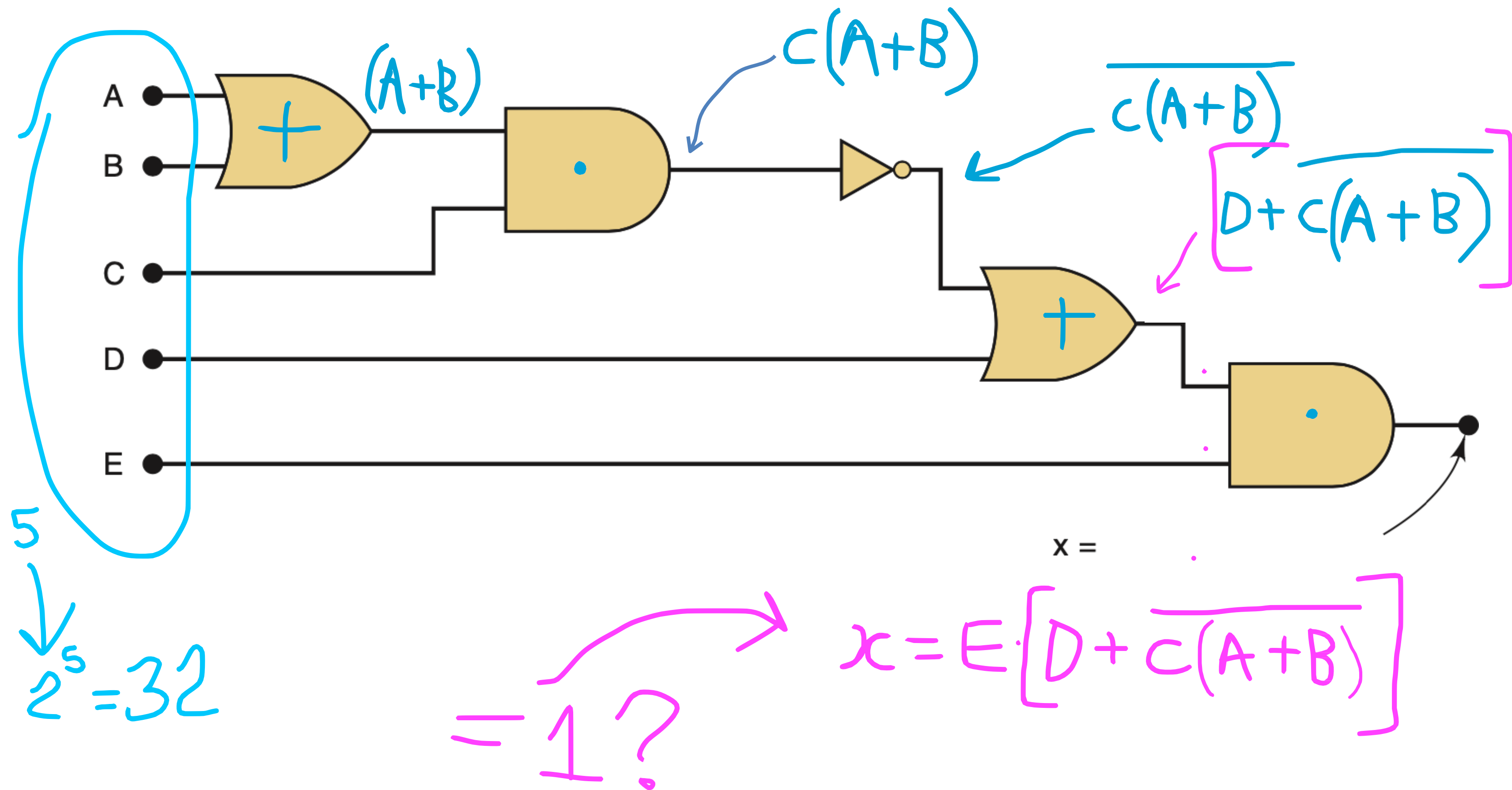
Deduzindo equações:



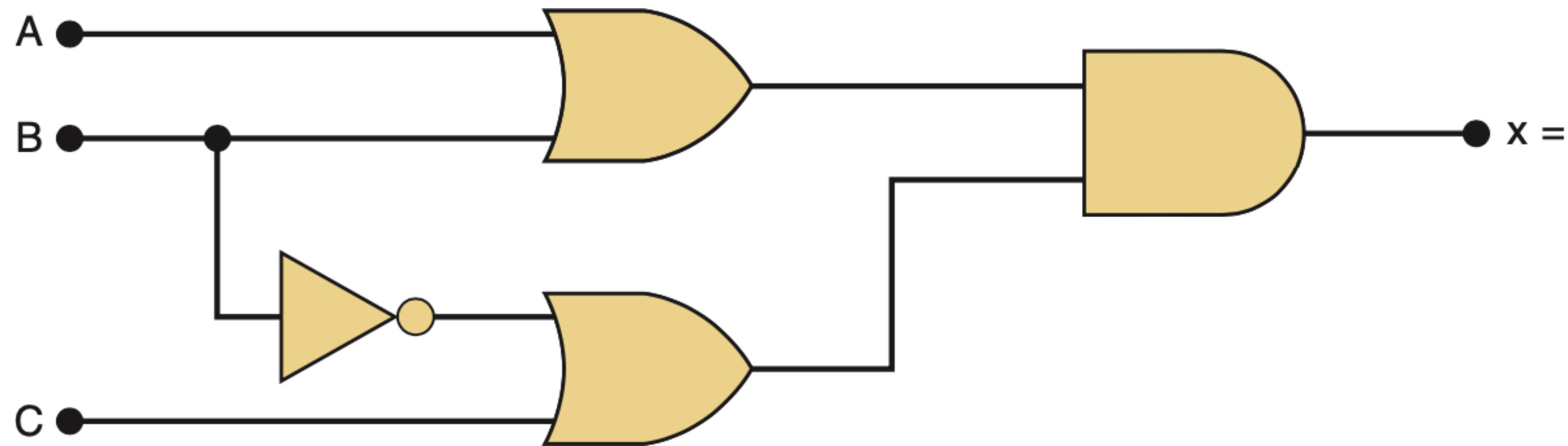
Deduzindo equações:



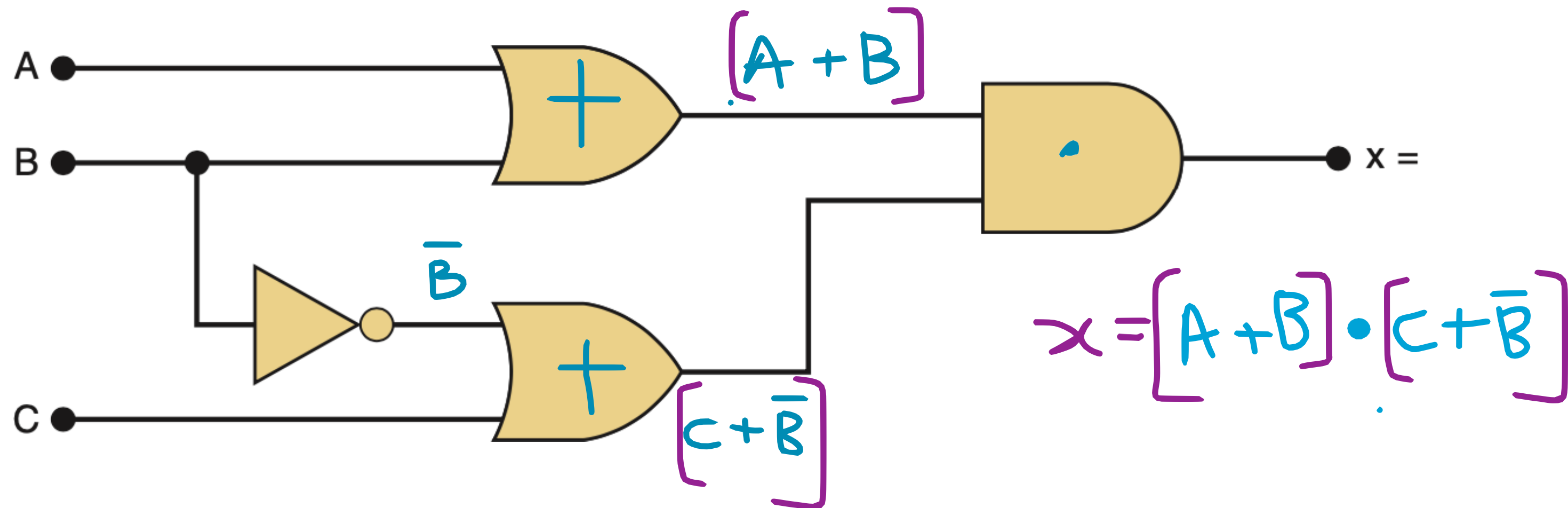
Deduzindo equações:



Deduzindo equações:



Deduzindo equações:

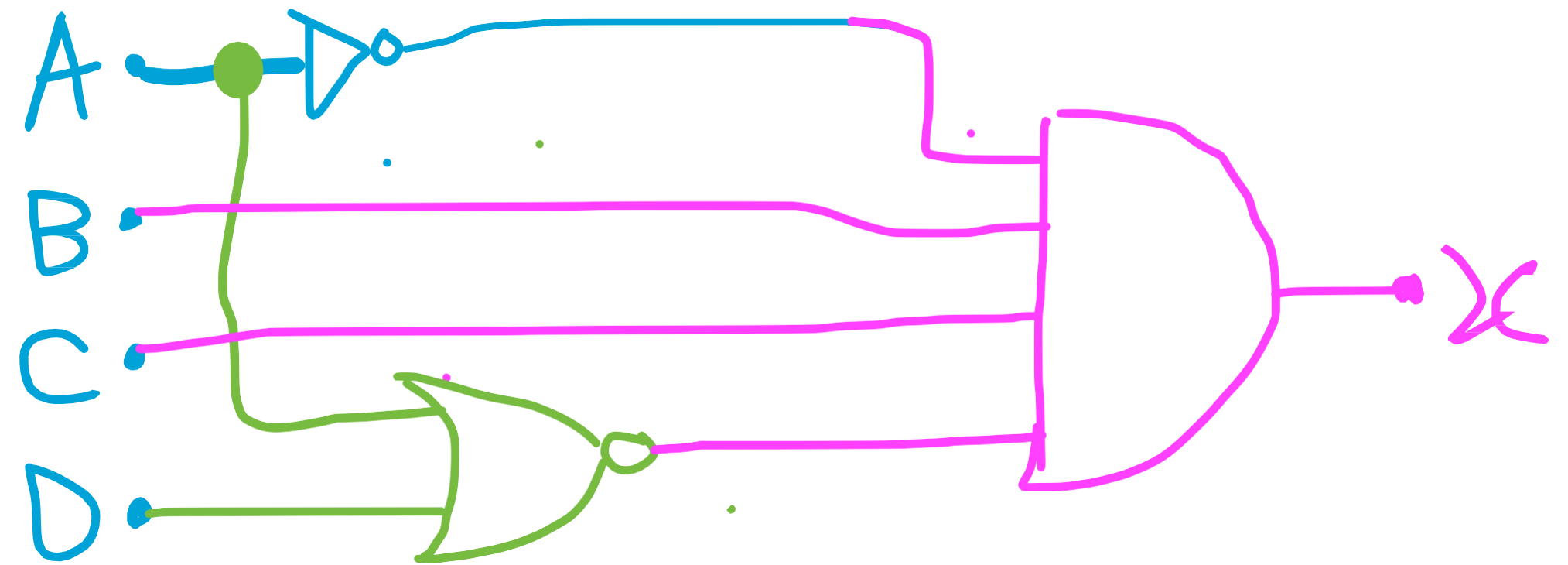


Desenhando circuitos:

- a) $x = \overline{A}BC \overline{(A + D)}$
- b) $y = AC + B\overline{C} + \overline{A}BC$
- c) $z = \overline{[D + (A + B)C]} E$

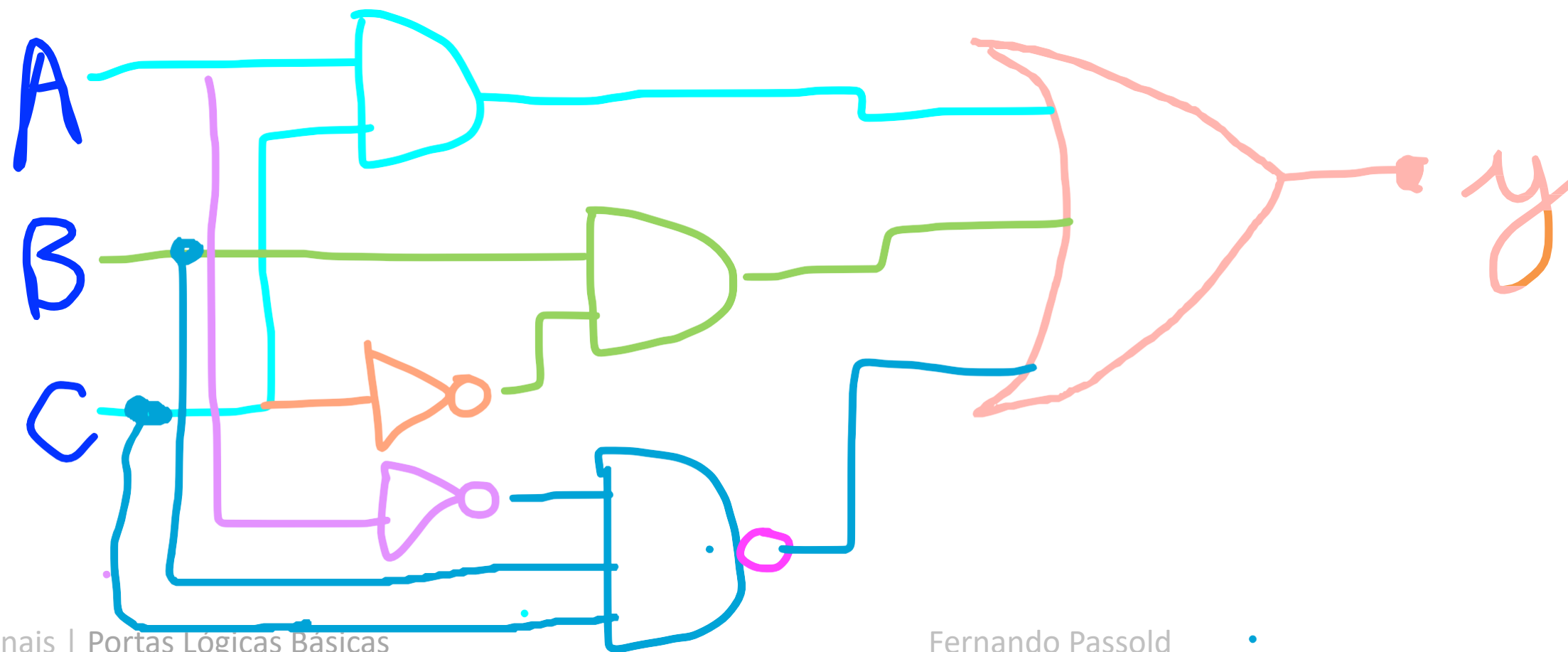
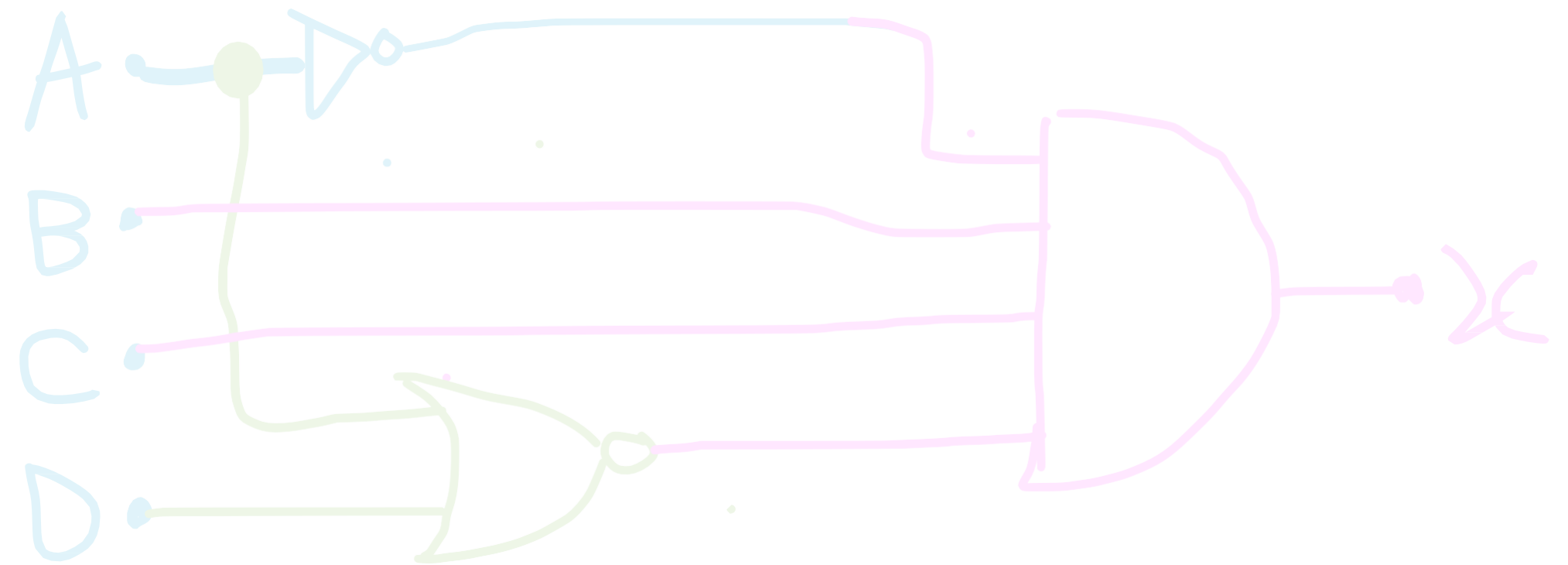
Desenhando circuitos:

- a) $x = \underline{\bar{A}BC} \cdot \underline{\overline{(A + D)}}$
- b) $y = AC + B\bar{C} + \overline{\bar{A}BC}$
- c) $z = \overline{[D + (A + B)C]} E$



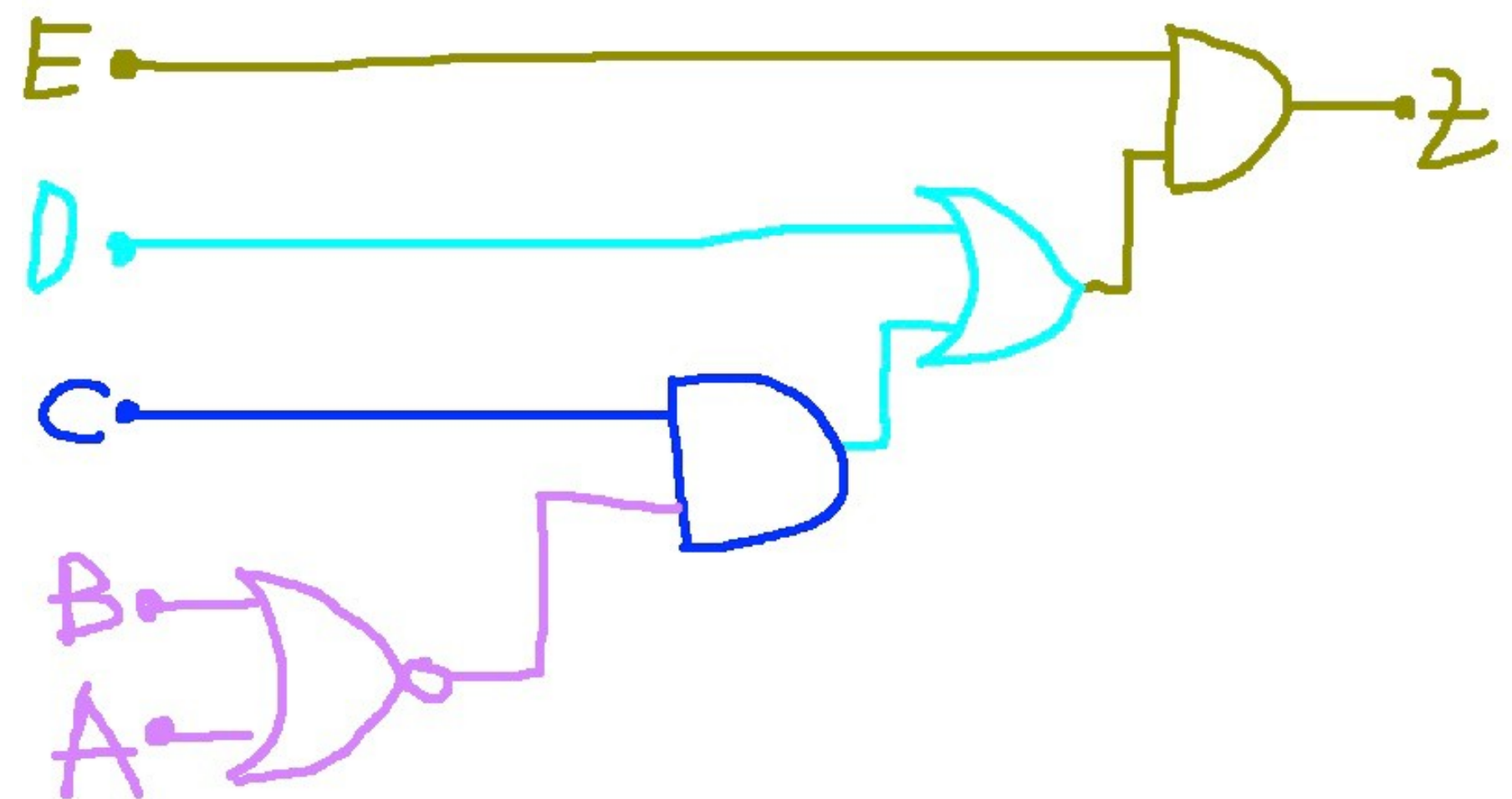
Desenhando circuitos:

- a) $x = \overline{A}BC \cdot \overline{(A + D)}$
- b) $y = \underline{AC} + \underline{B\overline{C}} + \underline{\overline{A}BC}$
- c) $z = [D + (A + B)C] E$



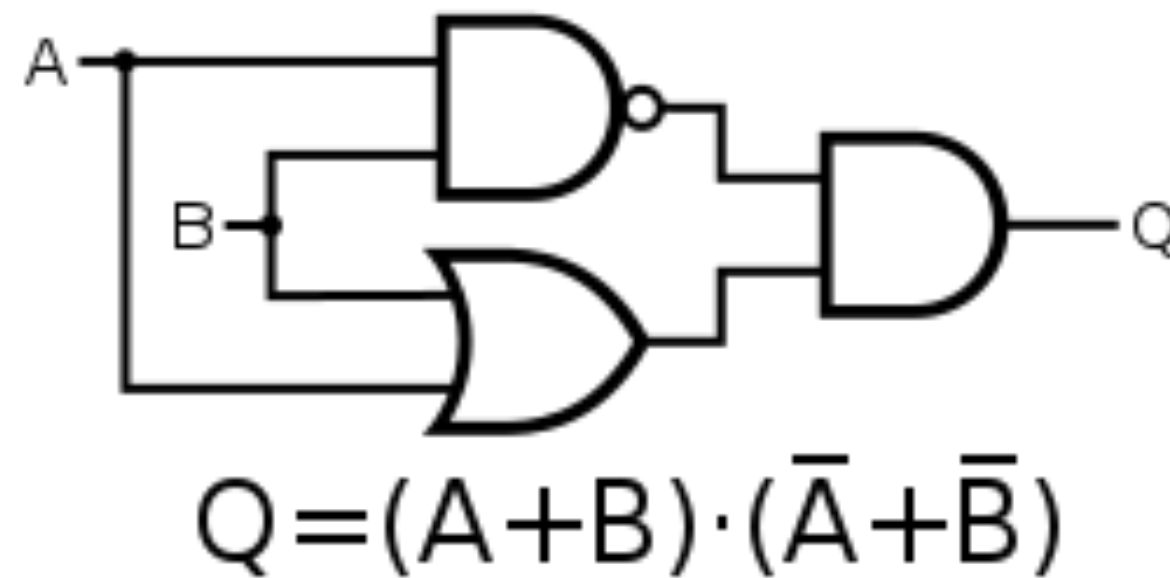
Desenhando circuitos:

- a) $x = \overline{A}BC \cdot \overline{(A + D)}$
- b) $y = \overline{A}C + B\overline{C} + \overline{A}BC$
- c) $z = \overline{[D + (A + B)C]} E$



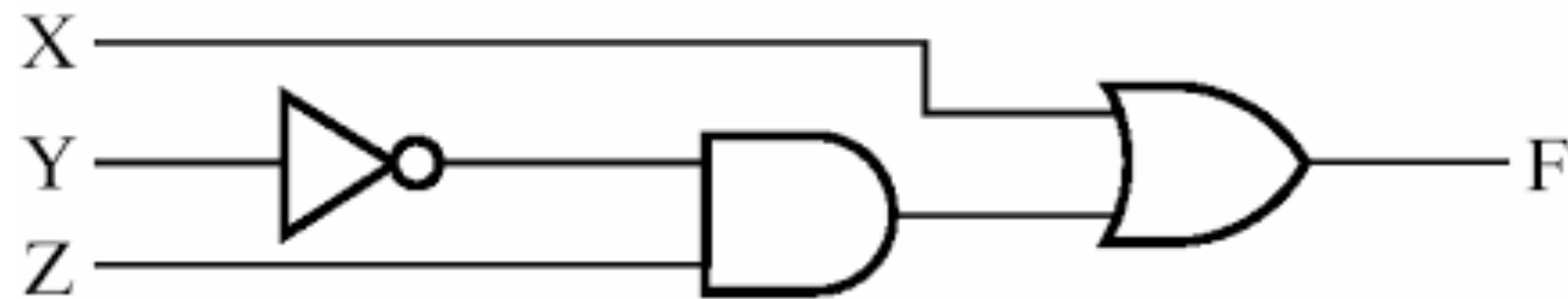
Deduzindo expressões e tabelas verdade

- Quando Q comuta para nível lógico ALTO?



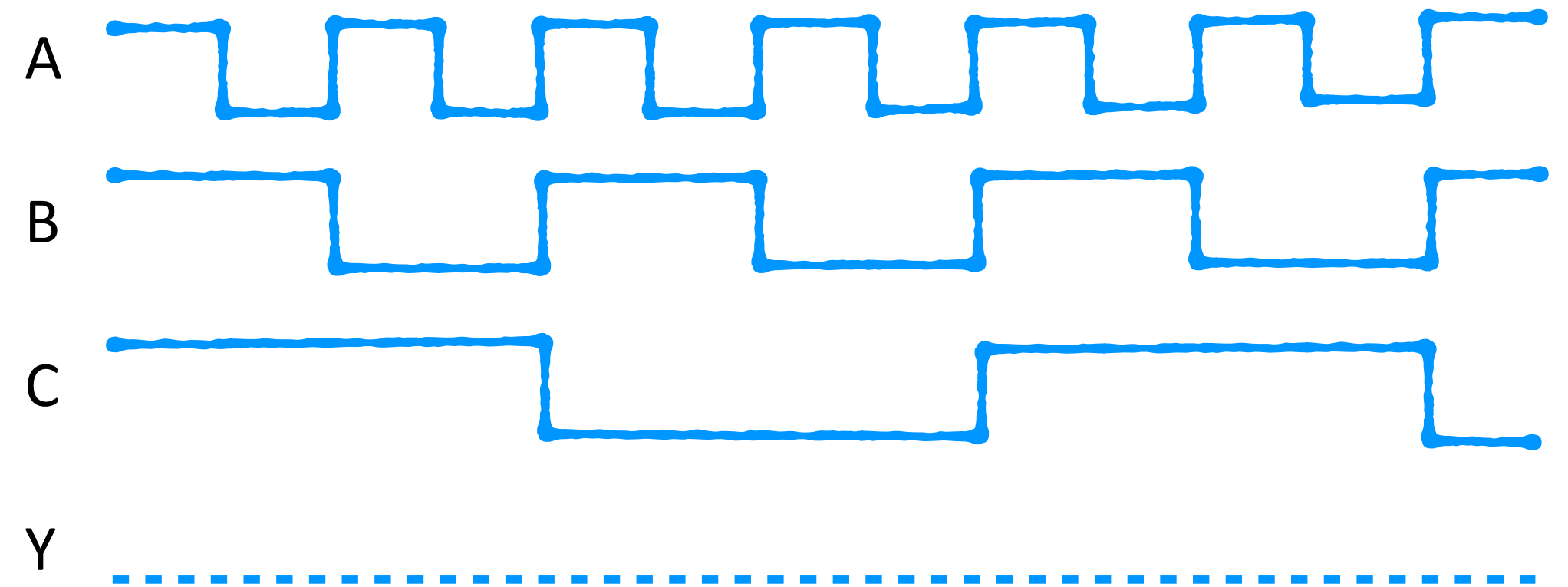
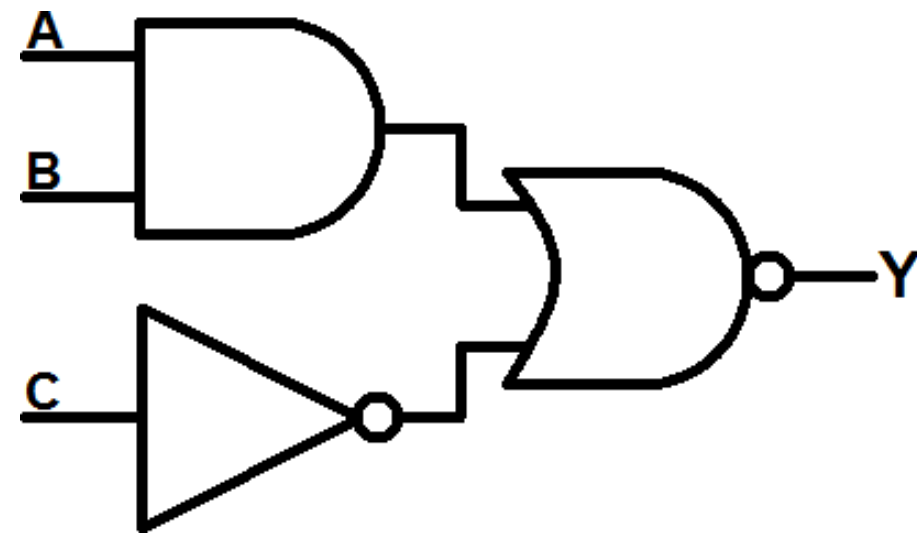
Deduzindo expressões e tabelas verdade

- Quando F comuta para nível lógico ALTO?



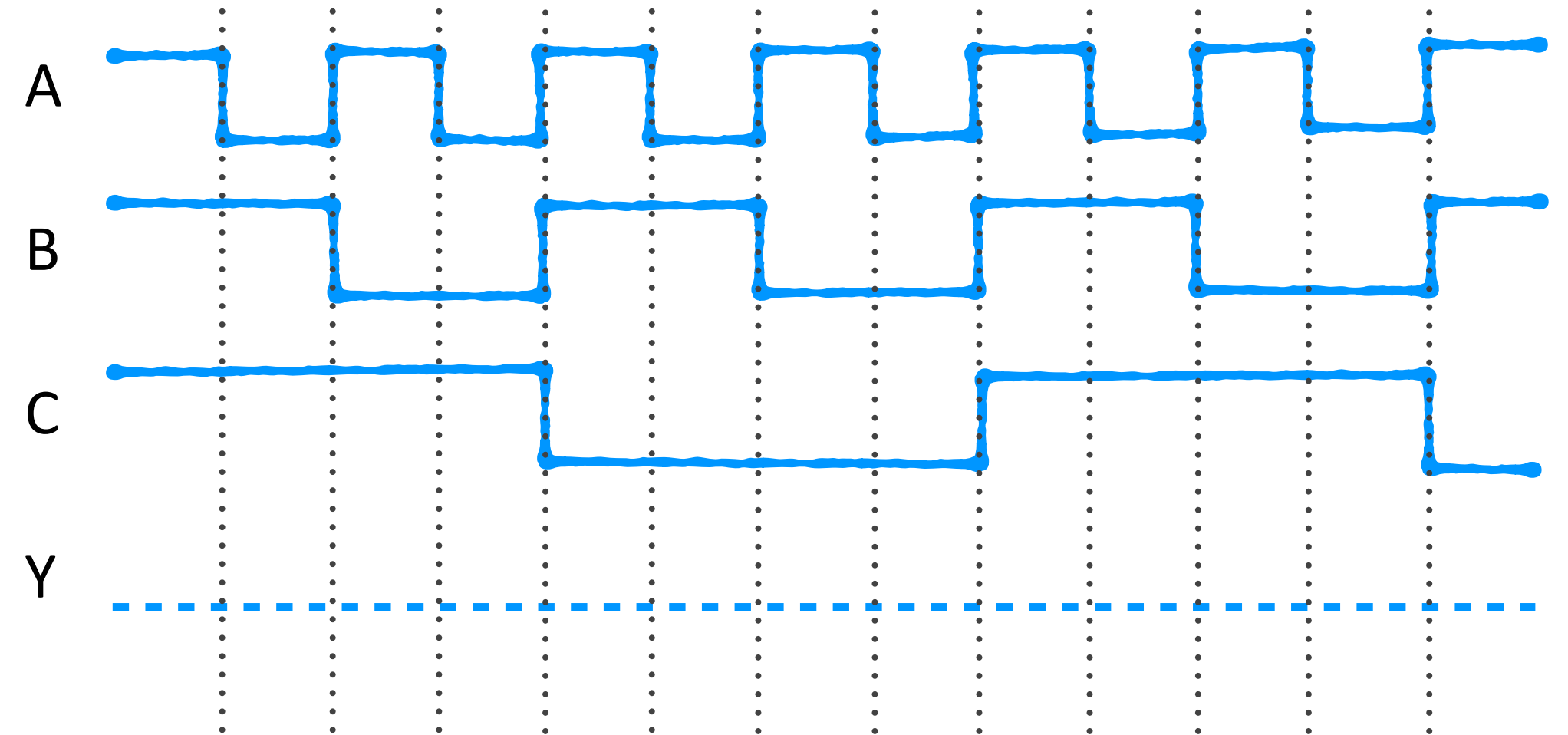
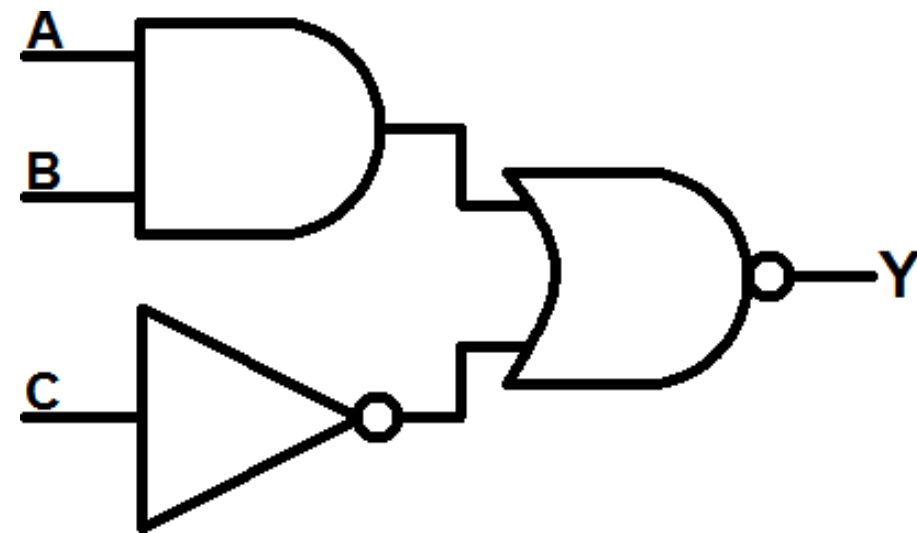
Deduzindo expressões e tabelas verdade

- Quando Y comuta para nível lógico ALTO?



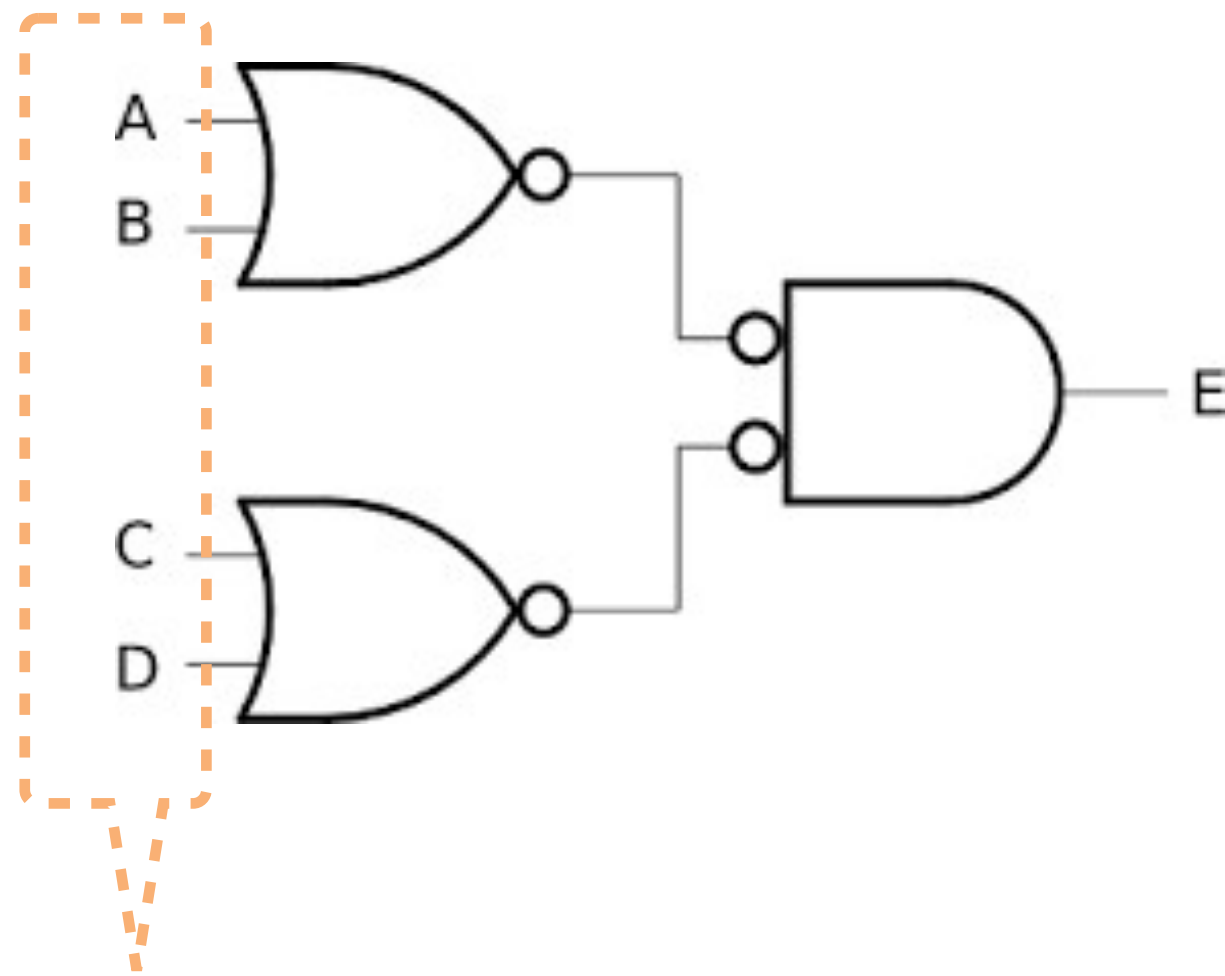
Deduzindo expressões e tabelas verdade

- Quando Y comuta para nível lógico ALTO?

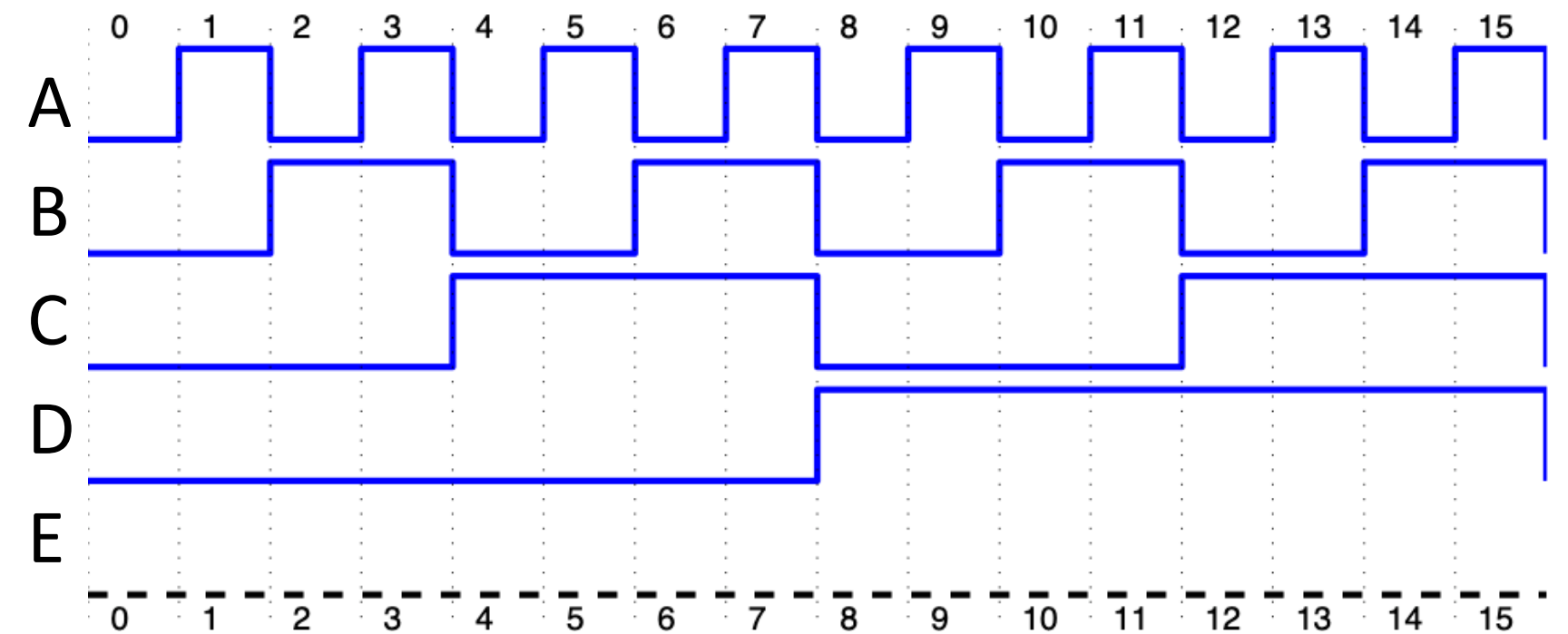


Deduzindo expressões e tabelas verdade

- Complete a forma de onda na saída E (deduza a tabela verdade para E):

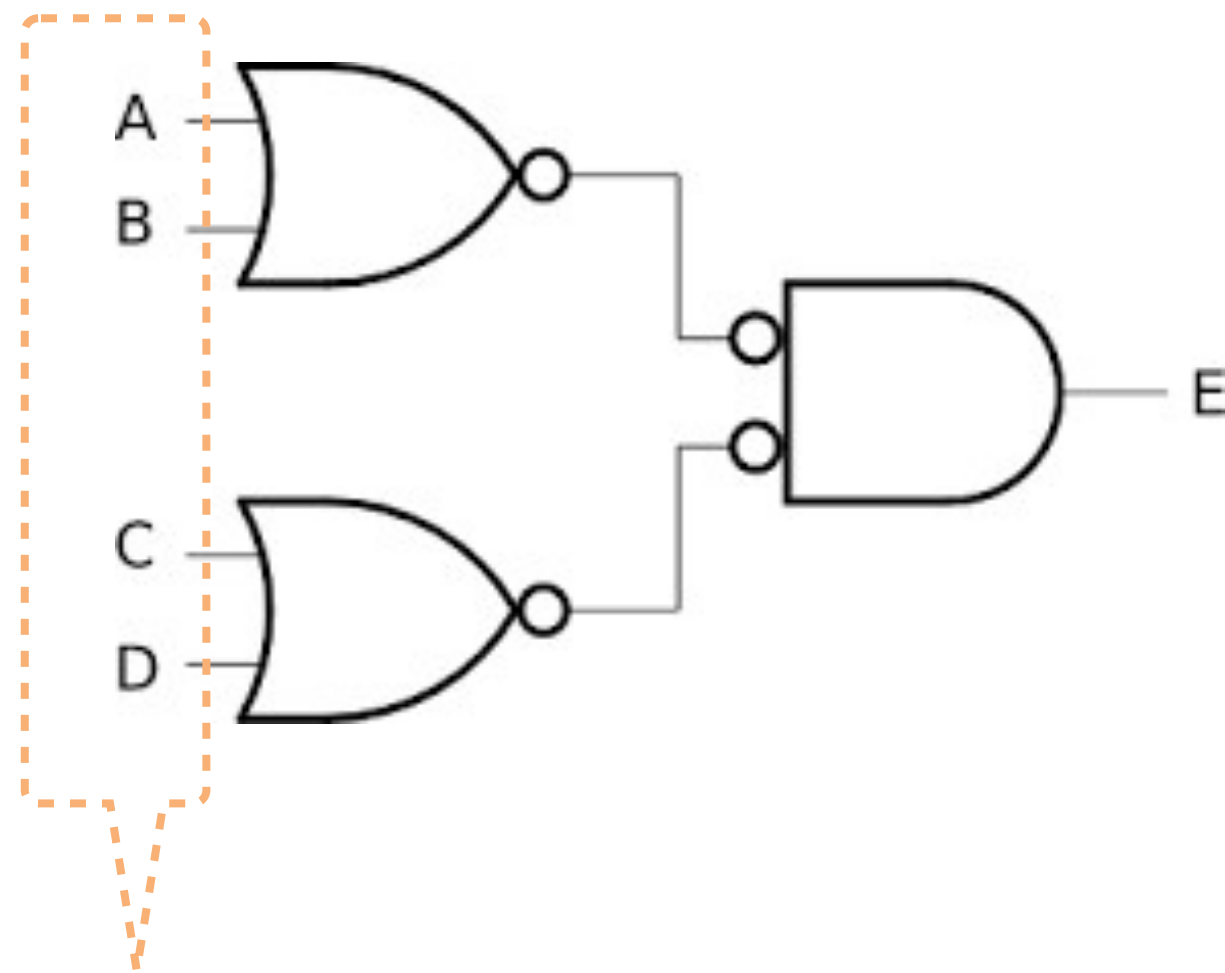


4 variáveis de entrada $\Rightarrow 2^4 = 16$ combinações de entrada

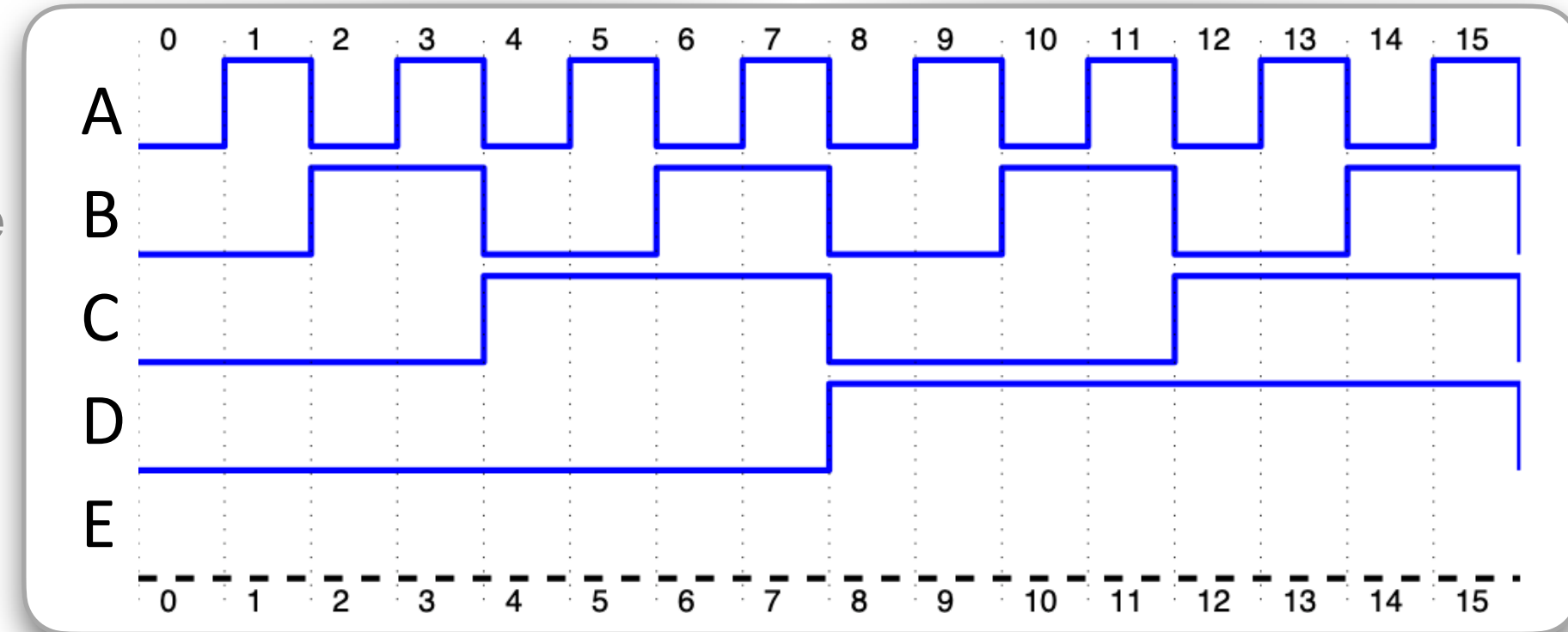


Deduzindo expressões e tabelas verdade

- Complete a forma de onda na saída E (deduza a tabela verdade)



4 variáveis de entrada $\Rightarrow 2^4 = 16$ combinações de entrada



Deduzindo expressões e tabelas verdade

- Quando Q comuta para nível lógico ALTO?

